IMPERIAL

Adaptive Acquisition: A Reinforcement Learning Framework for Autonomous MRI Sequence Optimization

Author

A. Allilaire

CID: 02372975

Supervised by

Dr Andreas Wetscherek
Prof. Wayne Luk

Second Marker

Prof. Ben Glocker

An Interim Report submitted in fulfillment of requirements for the degree of
Bachelor of Engineering in Computing

Department of Computing
Imperial College London

2026

Contents

1
Introduction

Contents

1.1 Introduction

In 2021, there were 395,000 cases of cancer in the UK, with 1 in 2 people in the UK being diagnosed with cancer in their lifetime. [1] Radiotherapy (RT) is a crucial component of cancer treatment; indeed, it is required for more than 50% of cancer patients. [2]

Image-guided radiation therapy (IGRT) is used to increase the precision of treatment and ensure accurate dose delivery. Conventionally, images are taken using a high-resolution X-ray technique called Cone Beam Computed Tomography. This is still the most common technique. [3]

However, poor image quality (caused by excessive scattering of X-ray photons) and ionising radiation are key drawbacks. Instead, Magnetic Resonance Imaging (MRI) can be used to improve soft tissue contrast (vital for liver or pancreatic cancer) and increase the accuracy of tumor target delineation. MR-guided radiation therapy (MRgRT) has been shown to significantly improve patient prognosis. [3]

MRI and a linear accelerator (the source of radiation) have been coupled into a hybrid Magnetic Resonance Accelerator (MR-Linac), the latest innovation in IGRT. The MR-Linac allows on-table adaptation of treatment plans, to respond to tumour movements and progression. However, one of the clinical bottlenecks is the extra time required for image acquisition. [4] The high-resolution images needed for accurate delineation require extended scan time. This also increases the risk of motion artifacts (inaccuracies due to patient movement).

There has been significant research into accelerating image acquisition, focusing mainly on using Deep Learning (computational neural network approaches) for image reconstruction (translating the raw MR signal into an image). Leading to significant improvements to image robustness, artifact reduction and image quality when using undersampled data.

There is now a growing interest in using DL and Reinforcement Learning (agent-based approaches) to develop novel pulse sequences, which are currently designed by MR experts. Novel pulse sequences have shown promise in speeding up image acquisition while maintaining image quality. [5]

1.2 Research challenges and achievements

This project studies whether reinforcement learning can be used to design adaptive quantitative MRI (qMRI) sequences. Rather than only reconstructing an image from a fixed acquisition, the aim is to let an agent choose the next acquisition based on what has already been measured. This makes the problem attractive, but also exposes three practical challenges.

C1: simulator validation for qMRI. The RL agent learns entirely from simulated MRI data, so the simulator must be numerically trustworthy before any learned policy can be evaluated. A visually plausible image is not sufficient: for qMRI, the pipeline must recover the correct quantitative tissue parameters, such as T_1 and T_2, from a known phantom.

C2: computational efficiency under expensive simulation. A live Bloch-equation simulator is much slower than the analytical environments normally used in reinforcement learning. This makes training compute-bound. The project therefore needs a multi-fidelity strategy: fast, lower-fidelity configurations for learning, with more accurate simulations reserved for validation.

C3: adaptive sequence design from tissue-parameter estimates. The final objective is not simply to run a fixed protocol faster, but to choose acquisitions based on the current estimate of the tissue parameters. The agent should use the signals observed so far to decide which inversion times, echo times, or flip angles are most informative next.

The report is organised around three corresponding achievements.

A1: a validated digital twin and simulator pipeline. The project built `MRISystemPhantom.jl`, a configurable digital twin of the QalibreMD MRI system phantom, and used it to validate the KomaMRI simulation, reconstruction, and fitting pipeline. This validation found two floating-point bugs in KomaMRI's long-sequence timing code, which were reduced to minimal reproducers and submitted upstream.

A2: a compute-aware, multi-fidelity RL environment. The project developed a Python/Julia Gymnasium environment that connects reinforcement learning agents to KomaMRI while managing the cost of simulation. The key design is to separate fast training configurations from higher-fidelity evaluation, so that policies can be trained within the available compute budget and then checked under more realistic simulation settings.

A3: adaptive qMRI sequence design. The project implements adaptive sequence-design experiments in which the agent selects acquisition parameters using the current tissue-parameter estimates. This links the validated simulator and multi-fidelity training loop to the central research question: whether learned adaptive protocols can outperform fixed conventional qMRI sequences.

2
Background & Related Work

Contents

It is necessary to understand the current process of MRI Pulse Sequence design before attempting to use DL or RL to improve upon it. This requires an understanding of MRI-Physics; a short introduction follows.

2.1 Principles of Magnetic Resonance Imaging

The signal generated by an MRI uses the Nuclear Magnetic Resonance (NMR) of protons. All subatomic particles have intrinsic angular momentum, which is a quantum property called spin. For example, a

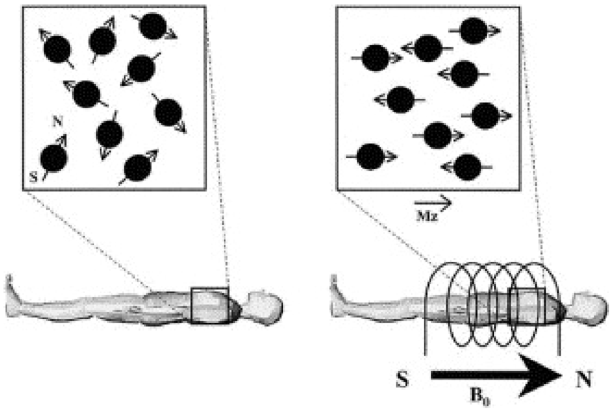


Figure 2.1: Protons align in parallel or antiparallel to the external field. Parallel alignment is a lower energy state, thus slightly more popular, leading to a net magnetisation vector (Mz). Figure from [7]

proton has a spin-$\frac{1}{2}$, while a neutron has a spin$\frac{1}{2}$. For a nucleus to have a net spin, it cannot have the same number of neutrons and protons.

A Hydrogen nucleus is made of a single proton, so it has a net spin-$\frac{1}{2}$. The angular momentum, combined with the mass and charge of a proton, results in a magnetic moment μ. When a uniform magnetic field (B_0) is applied, protons will align their magnetic axis with the field. (See Figure 2.1) However, a proton wobbles around the field's axis, which is called precession. Essentially, the proton behaves like a microscopic bar magnet rotating around its axis.[6]

The frequency of this rotation is known as the Larmor frequency (ω_0) and is directly proportional to the magnetic field (B_0). The constant of proportionality is called the gyromagnetic ratio (γ). For protons, $\gamma/2\pi = 42.58$ MHz T^{-1}. [7]

$$\omega_0 = \gamma B_0 \tag{2.1}$$

Here γ is the angular-frequency value. In the KomaMRI simulations later in this thesis, the cycle-frequency convention is used instead, with $\gamma = 42.58$ MHz T^{-1}. Accidentally using the angular-frequency value in this context introduces an extra factor of 2π, which shrinks the encoded field of view by 2π.

2.1.1 RF Pulses

The overall net magnetisation from all protons is represented by the vector $\mathbf{M} = (M_x, M_y, M_z)$. At equilibrium, M_z is aligned with the static B_0 field. The transverse components M_x and M_y, which rotate about B_0, are usually grouped into the complex notation $M_{xy} = M_x + iM_y$.

The receiver coils measure transverse magnetisation M_{xy}, not longitudinal magnetisation M_z directly. A perpendicular radio-frequency field B_1, applied at the Larmor frequency, tips some or all of M_z into the transverse plane. This creates M_{xy}, which precesses about B_0 and induces a voltage in receiver coils

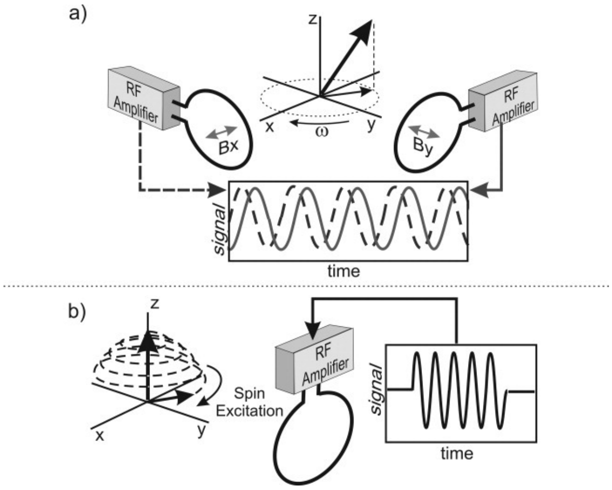


Figure 2.2: A voltage is induced via electromagnetic induction, making an MR signal. (A) The coils are perpendicular and can measure the phase and amplitude of the transverse magnetic vector. (B) RF pulses progressively "flip" the z-axis magnetisation vector into the transverse plane. Figure from [6]

placed perpendicular to the main field, as shown in Figure 2.2. The B_1 pulses are called radio-frequency (RF) pulses because the Larmor frequency lies in the radio-frequency range. [7][6]

The resulting angle between M and B_0 is called the flip angle (α) and can be calculated using:

$$\alpha = \gamma \int B_1(t)\, dt \tag{2.2}$$

2.1.2 T1, T2 & T2* Relaxation

After an RF pulse is switched off, M_z recovers towards thermal equilibrium along B_0, while M_{xy} decays in the transverse plane (the free induction decay). These are independent processes: longitudinal relaxation transfers energy back to the surrounding lattice, while transverse relaxation describes loss of phase coherence between spins.

Longitudinal, or spin-lattice, relaxation is characterised by T_1: the time taken for M_z to recover 63% of the way back to its equilibrium value M_0 after excitation. Because this recovery depends on energy exchange with the local molecular environment, T_1 is an intrinsic tissue parameter.[7]

Transverse, or spin-spin, relaxation describes the decay of M_{xy} as spins lose phase coherence after excitation. Irreversible dephasing from spin-spin interactions is characterised by T_2, while reversible dephasing from static B_0 inhomogeneity is written as T_2'. The observed transverse decay T_2^* combines both effects:

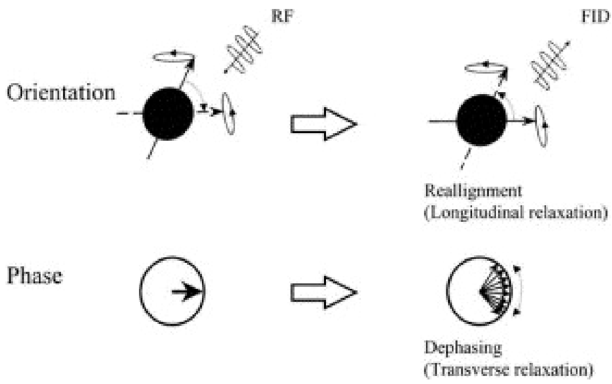


Figure 2.3: **Upper Row:** Longitudinal relaxation (decreasing the flip angle).
Lower Row: Transverse relaxation (spin dephasing) Figure from [7]

$$\frac{1}{T_2^*} = \frac{1}{T_2} + \frac{1}{T_2'}. \tag{2.3}$$

A 180° refocusing pulse reverses the static dephasing term T_2', which is why spin-echo sequences measure a signal governed primarily by T_2 relaxation. In biological tissue, T_2 is generally shorter than T_1.[6]

Bloch-equation model. The Bloch equations formalise the relaxation behaviour above. In the rotating frame at ω_0, with equilibrium magnetisation M_0 along $+z$, the relaxation dynamics can be written as:

$$\frac{dM_z}{dt} = -\frac{M_z - M_0}{T_1}, \qquad \frac{dM_{xy}}{dt} = -\frac{M_{xy}}{T_2} + i\Delta\omega M_{xy} \tag{2.4}$$

Here $\Delta\omega$ is the local off-resonance frequency relative to the rotating frame. The first equation describes longitudinal recovery: M_z exponentially returns to M_0 with time constant T_1. The second describes transverse decay: the magnitude of M_{xy} falls with time constant T_2, while off-resonance produces additional phase rotation.

Between RF pulses, these independent differential equations have the free-precession solutions:

$$M_z(t) = M_0 + (M_z(0) - M_0)\, e^{-t/T_1}, \qquad |M_{xy}(t)| = |M_{xy}(0)|\, e^{-t/T_2} \tag{2.5}$$

These are the signal-decay relationships used by the conventional quantitative MRI fitting methods used later in this work. Under the hard-pulse approximation, an RF pulse is treated as an instantaneous rotation by flip angle α about a transverse axis. If the longitudinal magnetisation immediately before excitation is M_z^-, the pulse leaves $\cos(\alpha)M_z^-$ on the longitudinal axis and tips $\sin(\alpha)M_z^-$ into the transverse

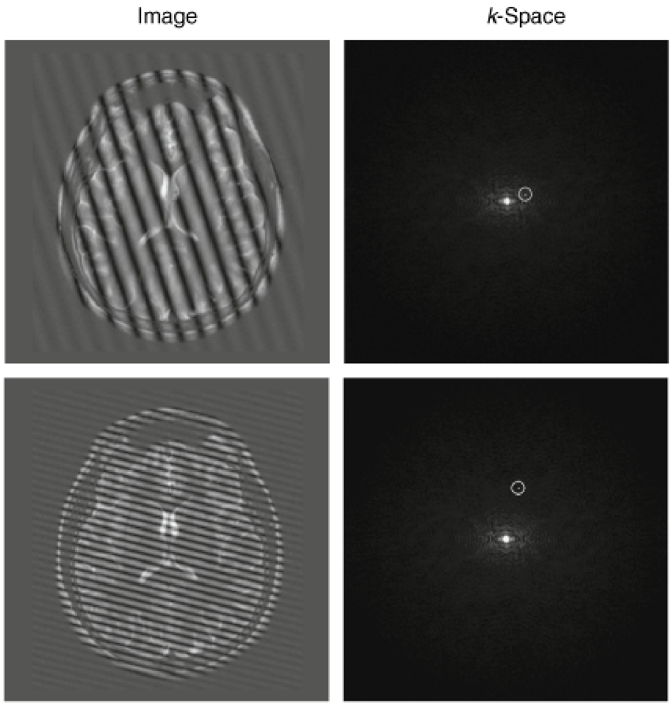


Figure 2.4: Image space and k-space are Fourier-pair representations of the same data: an MR image can be viewed either as a matrix of intensities $I(x, y)$ or as spatial-frequency amplitudes $S(k_x, k_y)$. Each k-space point weights a sinusoidal wave across the field of view; the vector angle (k_x, k_y) sets the wave-pattern orientation, and its distance from the origin sets the spatial frequency. The centre of k-space contains low-spatial-frequency contrast and mean signal, while the outer samples contain high-spatial-frequency detail. Figure from [8].

plane. The received signal is proportional to this transverse component.

After readout, the sequence models used in this work assume that residual transverse magnetisation is removed by gradient spoiling or by relaxation before the next acquisition block. The only state carried between shots is therefore the scalar longitudinal magnetisation, updated by RF rotations and T_1 recovery. This reduces the Bloch dynamics to simple recurrence relations; without this assumption, transverse coherences would need to be tracked explicitly using a coherence-pathway method such as Extended Phase Graphs (EPG).

2.1.3 Spatial Encoding & K-Space

Relaxation determines how much signal a spin can produce, but an image also requires spatial localisation. MRI does not measure image pixels directly. Instead, the scanner samples k-space: a spatial-frequency representation of the image, shown in Figure 2.4. The centre sample of k-space, $S(0, 0)$, has no gradient moment along either axis, so all spins contribute without a spatial phase ramp. It is therefore the integral of the whole image, or mean signal, conventionally called the DC term.

Let G_x, G_y, and G_z denote magnetic-field gradients along the scanner axes. These gradients make the

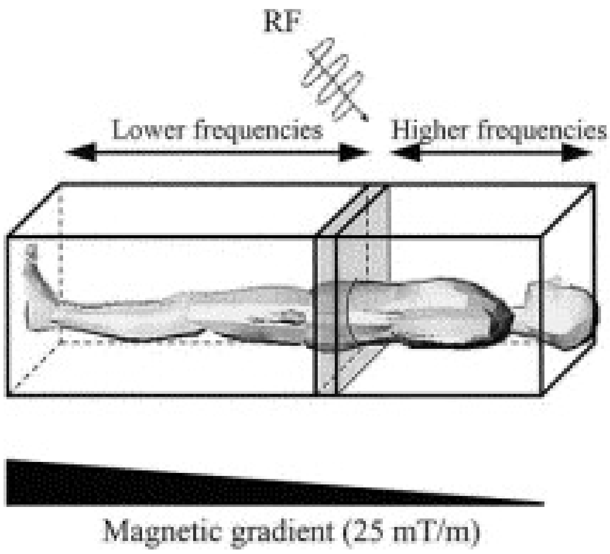


Figure 2.5: Slice selection using a narrow-band RF pulse applied during a gradient. Only spins whose shifted Larmor frequencies lie within the RF bandwidth are excited. Figure from [7].

Larmor frequency position-dependent by varying the local field strength, providing the spatial information missing from the raw MR signal.

The G_z gradient performs slice selection. During the RF pulse, G_z maps z-position to Larmor frequency, so a narrow-band RF pulse excites only the spins whose frequencies lie within the RF bandwidth. This selects a thin slice of the object, as shown in Figure 2.5.

The in-plane coordinates are then encoded using G_y and G_x. A short G_y pulse before readout provides phase encoding: it gives spins at different y-positions different accumulated phases. A G_x gradient is kept on during ADC readout for frequency encoding: successive samples trace spatial frequency along k_x. Repeating the sequence with different G_y amplitudes steps through k_y, so each repetition fills a different row of k-space.

For a gradient waveform $\mathbf{G}(t)$, the sampled k-space location is determined by the accumulated gradient moment:

$$\mathbf{k}(t) = \gamma \int_0^t \mathbf{G}(\tau)\, d\tau. \tag{2.6}$$

Stronger or longer gradients move the acquisition further through k-space. At a given k-space location, the acquired signal is

$$S(\mathbf{k}) = \int m(\mathbf{r}) e^{-i2\pi\mathbf{k}\cdot\mathbf{r}}\, d\mathbf{r}. \tag{2.7}$$

Here $m(\mathbf{r})$ is the transverse magnetisation at position $\mathbf{r}$. The exponential term applies a position-dependent phase rotation to each complex M_{xy} contribution before the receiver sums over the object. Changing $\mathbf{k}$ changes this phase pattern, so each measurement extracts a different spatial-frequency component of the image. The image is recovered by applying an inverse Fourier transform to the sampled k-space.

Two design quantities follow directly from this sampling geometry. The k-space sample spacing determines the field of view: $\text{FOV} = 1/\Delta k$. If the object extends beyond this FOV, it aliases back into the image. The k-space extent determines the nominal pixel size, $\Delta x = \text{FOV}/N \approx 1/(2k_{\text{max}})$. Acquiring a wider k-space window improves spatial resolution, but it requires more samples. In particular, higher k_y resolution means acquiring more phase-encode rows, which usually requires additional sequence repetitions.

FFT ordering. The Fourier relationship above defines the k-space/image-space mapping, but not the array ordering used to store samples. KomaMRI returns Cartesian ADC data in acquisition order, with the DC sample $S(0, 0)$ near the centre of the stacked k-space matrix. For an even matrix size N, the standard acquisition convention samples one extra negative line,

$$-N/2,\ -N/2+1,\ \ldots,\ 0,\ \ldots,\ N/2-1,$$

so for $N = 32$ this is $-16, \ldots, 0, \ldots, 15$. DC therefore lies at index $N/2 + 1$ in one-based indexing. Standard FFT routines instead expect DC at the first array index, so reconstruction requires

$$I = \text{fftshift}\left(\text{ifft}\left(\text{ifftshift}(S)\right)\right),$$

Here ifftshift converts the centre-DC acquisition ordering to FFT ordering, and fftshift recentres the reconstructed image. Without these shifts, the image is reconstructed under the wrong indexing convention, causing a half-FOV shift and possible checkerboard phase modulation.

2.1.4 Gibbs (Truncation) Ringing

Finite k-space support. The Fourier relationship is exact only if all of k-space is measured. In practice, the acquisition covers a finite $N_{pe} \times N_{fe}$ rectangle. This is equivalent to multiplying the ideal infinite k-space by a rectangular window. By the convolution theorem, multiplication in k-space becomes convolution in image space:

$$I_{\text{recon}}(\mathbf{r}) = I_{\text{true}}(\mathbf{r}) * \text{PSF}(\mathbf{r}). \tag{2.8}$$

The point-spread function (PSF) is the Fourier transform of the sampling window. For a one-dimensional rectangular window, it is the Dirichlet kernel,

$$\text{PSF}(x) = \frac{\sin(\pi N x/\text{FOV})}{N \sin(\pi x/\text{FOV})}. \tag{2.9}$$

This PSF has a narrow main lobe but slowly decaying side-lobes. At sharp intensity transitions, such as the water–sphere boundaries in the phantom, those side-lobes do not cancel and appear as Gibbs ringing. Increasing N makes the ringing finer in physical units, but it does not remove the side-lobes; the artefact is caused by the rectangular truncation itself. The usual suppression method is apodisation, such as a Hamming window, which down-weights outer k-space to reduce ringing at the cost of a wider PSF and therefore slightly lower spatial resolution. Appendix A.5 derives this trade-off, since it is used directly by the phantom library's Hamming reconstruction option.

2.1.5 Pulse Sequences

A pulse sequence is a description of a series of RF pulses and MR-signal acquisitions. These are "hand-crafted by MR experts, who combined strong intuitions and an understanding of spin physics with mathematical solutions of the Bloch equations"[5]. They try to maximise signal-to-noise ratio (SNR) and image contrast while reducing scan times. To do this, they optimise parameters such as the echo time (TE) and repetition time (TR).

For example, the time between two RF pulses is called the repetition time.

2.1.6 MRI Simulators & Pulseq

Pulseq is a standardised, open-source framework for defining MRI pulse sequences. Pulseq is hardware-agnostic (compatible with all MR-Linacs).[9]. Pulseq can also be used with many MRI simulators, including both KomaMRI and MRZero.

KomaMRI is a high-performance Julia-based MRI simulator which enables scalable, multi-threaded computation to speed up simulations. [10] MRZero is a "PyTorch-based framework for easy MRI sequence optimisation and development of self-learning sequence development strategies". [11]

2.1.7 Magnetic Resonance Fingerprinting (MRF)

Conventional MRI predominantly produces qualitative, “weighted” images where pixel intensity represents a relative contrast. This depends on acquisition parameters such as coil sensitivity, and makes it difficult to standardise across different scanners. In contrast, Quantitative MRI (qMRI) aims to measure the fundamental physical properties of tissues, such as the longitudinal (T_1) and transverse (T_2) relaxation times mentioned earlier. A significant advancement in this field is Magnetic Resonance Fingerprinting (MRF), which replaces traditional steady-state sequences with ”pseudorandomized acquisition parameters”. By varying the flip angles and repetition times (T_R) throughout the scan, MRF generates a unique signal evolution—or “fingerprint”—for every tissue type. These signals are then pattern-matched against a pre-computed dictionary of Bloch equation simulations, allowing for the simultaneous and rapid mapping of multiple tissue properties in a single acquisition. For MR-guided radiotherapy, this quantitative approach offers the potential for “biological gating”, where treatment adaptation is driven by functional changes in tumour physiology rather than just anatomical shifts. [12] [13]

2.2 Magnetic Resonance Linear Accelerator (MR-LINAC)

The MR-LINAC represents a paradigm shift in radiation oncology, moving beyond conventional IGRT, which provides only a pre-treatment ’snapshot’ of the patient’s anatomy. Instead, it enables a continuous feedback loop between real-time imaging and dose delivery, allowing for dynamic adjustment during the treatment itself, known as Adaptive Radiotherapy (ART).

2.2.1 Clinical Benefits of ART

By capturing high-contrast images while the patient is on the treatment table, clinicians can account for ”inter-fraction” changes (variations between daily sessions) and ”intra-fraction” motion (movement during the session, such as breathing or bowel movements).

This real-time visualisation allows for a significant reduction in the Planning Target Volume (PTV). Traditionally, large margins are added to the Clinical Target Volume (CTV) to compensate for uncertainties in organ position. Using MRgRT, these margins can be tightened. In prostate cancer, PTV margins have been reduced to 2mm, significantly reducing the volume of radiated healthy tissue. Further, better soft tissue contrast is essential to accurately delineate the tumour and contour Organs at Risk (OaR) in complex regions like the liver and pancreas.

This allows for ”hypofractionation” (delivering a more potent dose over fewer sessions) without increasing the toxicity profile for the patient. This significantly decreases treatment side effects. [14]

2.2.2 Current Limitations and Implementation Challenges

Despite its potential, the global adoption of MR-LINAC technology remains in its early stages. As of 2024, four primary operational configurations exist, typically utilising magnetic field strengths of either 0.35T (e.g., ViewRay MRIdian) or 1.5T (e.g., Elekta Unity) [14]. The transition to MRgRT introduces several operational bottlenecks:

- **Complexity and Workflow:** The requirement for “on-table” plan adaptation demands a high level of expertise and significant upskilling for radiation therapists, physicists, and oncologists.
- **Delineation Uncertainty:** While tumours frequently shrink during treatment, it is currently unclear if adapting to these smaller volumes is always safe, as the original tumour bed may still contain microscopic disease [4].
- **Time Constraints:** Re-tuning treatment plans while the patient is immobilised requires rapid image acquisition and contouring to prevent motion artifacts and patient discomfort.

2.2.3 The Role of AI and Quantitative MRI

To address the clinical bottleneck of manual contouring, AI-based automated segmentation has emerged as a vital tool. Tools such as the Philips RTdrive Core have demonstrated clinical robustness in prostate cancer patients, significantly reducing the workload for clinicians by automatically outlining the target and OaRs. [15].

Furthermore, the future of MRgRT lies in moving beyond simple anatomical contrast. The integration of quantitative MRI (qMRI), such as measuring tissue cell density through Diffusion-Weighted Imaging, allows for biological targeting. By applying Deep Learning and Reinforcement Learning (RL) to these multi-parametric datasets, the MR-LINAC could eventually provide an end-to-end automated system that optimises dose delivery based on the functional response of the tumour in real-time [4].

2.3 Deep Learning in MRI

Long acquisition times represent a major barrier in clinical MRI, primarily due to the inherent trade-off between image quality and speed of imaging. Indeed, the 3D images crucial for tumour delineation

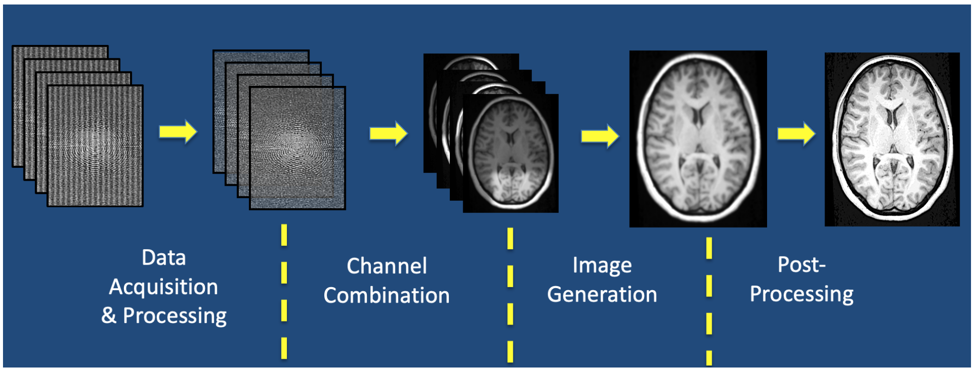


Figure 2.6: Courtesy of Allen D. Elster, MRIquestions.com. [16]

require longer acquisition times, which increases the risk of motion artifacts. There is a large body of research into reducing acquisition time while maintaining image quality. Notable breakthroughs include Parallel Imaging (multiple receiver coil arrays) and Compressed Sensing. Unfortunately, Parallel Imaging quickly reaches practical Signal-to-Noise Ratio (SNR) limits, while compressed sensing is prohibitively computationally expensive. [5]

To supplement, DL has been widely applied, both in research and through commercial products, to reduce imaging time and improve image quality. For example, DL is used in post-processing to remove noise, improve image resolution and reduce image inaccuracies (called artifacts) from patient movement or field inhomogenities (e.g. caused by metal patient implants or pacemakers). [16]

2.3.1 Image Acquisition and Reconstruction using Deep Learning

Post-processing operates only on the output image, which allows leveraging traditional imaging techniques (CNN, image kernels) as well as being vendor-neutral (so hardware-agnostic). However, not having the raw signal limits the information available.

This has given rise to more complex networks that replace both the Fast Fourier Transform algorithm used for reconstruction and post-processing to create higher quality images.

2.3.2 Pulse sequence design

Recent research has explored the use of artificial intelligence to optimise the underlying physics of MR acquisition through the autonomous design of pulse sequences. Two differing approaches have been tested: an RL game-based approach and a more traditional supervised learning approach.

The MRzero framework introduces a supervised learning approach that treats the entire scanning and reconstruction pipeline as a fully differentiable system. By simulating RF events, gradient moments,

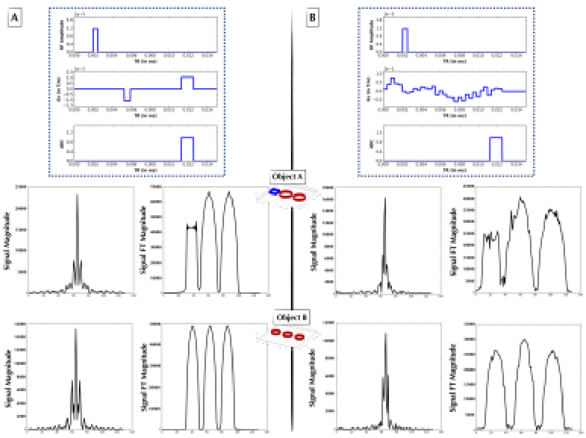


Figure 2.7: (B) Novel pulse sequence successfully developed by AUTOSEQ to mimic a gradient echo sequence. Figure from [17]

and delay times through a Bloch equation layer, the model can be optimised directly against a target contrast of interest (e.g. a T_1 contrast map). The cost function that was optimised by the model took into account: data fidelity, Specific Absorption Rate (SAR) penalties and scan time. The researchers successfully demonstrated that MRzero could ”learn” to replicate conventional gradient-echo sequences from scratch. This was then successfully tested on a clinical 3T scanner for in vivo human brain imaging. This demonstrates that deep learning-derived sequences are clinically viable and can be optimised for specific hardware constraints.[11]

By recasting pulse sequence development as a “game of perfect information,” researchers have implemented a reinforcement learning (RL) framework where an AI agent interacts with a Bloch equation-based physics simulator to generate novel sequence actions. This approach utilises a Bayesian derivative of RL, employing Gaussian processes and Bayesian neural networks to navigate the non-linear dynamics of nuclear magnetisation and maximise an acquisition function based on image reconstruction accuracy. Preliminary 1-D experiments demonstrate that such agents can independently “discover” canonical sequences, such as the gradient echo, and generate novel pulse sequences that approximate Fourier spatial encoding. While this was a 1-D experiment, further exploration could yield novel pulse sequences that reduce acquisition time and maximise SNR, alleviating the current acquisition bottleneck faced by the MR-Linac.[17]

2.3.3 Adaptive MRI acquisition

The above work focuses on the design of novel, but ultimately static pulse sequences. Instead, Adaptive MR is a promising new research direction. Here, a probability distribution of the tissue parameter of interest (e.g. T_2) is continuously updated with each new reading. The next pulse sequence is chosen based on the estimate of T_2. This has been shown to accelerate acquisition time by a factor of 2.5. This is a general paradigm shift in pulse sequence design that motivates and validates further exploration of adaptive MR.[18]

Adaptive MR also has the potential to be combined with RL to autonomously control in real-time MRI hardware. Walker-Samuel demonstrated this by using a Deep Deterministic Policy Gradient (DDPG) algorithm to control an MRI simulation environment where the acquisition process had been turned into a reward-based game. The RL agent was tasked with identifying phantom shapes while being penalised for increased acquisition time. This forces the system to optimise the trade-off between Signal-to-Noise Ratio (SNR) and image acquisition time. Notably, the agent autonomously learned to perform ”active sensing,” acquiring only four to six outer lines of k-space and dynamically adjusting T_E, T_R, and flip angles to act as an edge detector. With a reported shape-classification accuracy of 99.8%, this research suggests that RL-controlled scanners could move beyond human-readable imaging to perform high-speed, task-specific ”autonomous sensing”. However, this was purely simulation-based and a very simple task, whether this method can be applied to more complex situations is unproven.

2.3.4 Reinforcement Learning and PPO for Adaptive Acquisition

Reinforcement learning (RL) studies problems where an agent improves its behaviour by interacting with an environment. At each step the agent observes the current state of the environment, chooses an action, and receives a scalar reward. The objective is not to predict a labelled target directly, as in supervised learning, but to learn a policy that selects actions leading to high cumulative reward.[19]

The standard mathematical model is a Markov decision process (MDP),

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \rho_0, \gamma), \tag{2.10}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s_{t+1} \mid s_t, a_t)$ is the transition model, R is the reward function, ρ_0 is the initial-state distribution, and $\gamma \in [0, 1]$ is the discount factor. A trajectory is a sequence

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots). \tag{2.11}$$

In this project, the environment is the simulated adaptive qMRI acquisition loop. A state contains the information available to the agent so far, such as the current acquisition history and the current estimate of the phantom parameters. An action specifies the next acquisition choice, for example timing and RF parameters such as inversion time, echo time, repetition time, and flip angle. The simulator then returns reconstructed measurements and a reward based on whether the new acquisition improves the quantitative parameter estimate, typically measured through reduction in T_1 error.

For a stochastic policy $\pi_\theta(a \mid s)$ with parameters θ, the return from time t is

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}. \tag{2.12}$$

The RL objective is to choose policy parameters that maximise expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G_0]. \tag{2.13}$$

The expectation is over trajectories generated by the current policy interacting with the environment. This is important in simulation-in-the-loop MRI: the training data are not a fixed dataset. They are samples produced by repeatedly running the current policy, simulating the resulting sequence, reconstructing the image, fitting the tissue parameters, and computing the reward.

Policy gradients

Policy-gradient methods optimise $J(\theta)$ directly.[20], [21] The probability of a trajectory under a policy is

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1} \mid s_t, a_t) \pi_\theta(a_t \mid s_t). \tag{2.14}$$

The simulator dynamics P do not depend on the policy parameters. Applying the log-derivative trick gives the basic policy-gradient expression:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t \right]. \tag{2.15}$$

In practice this expectation is approximated using sampled rollouts:

$$\hat{g} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \hat{A}_{i,t}. \tag{2.16}$$

This equation is the central sample-based approximation used by policy-gradient RL. Instead of knowing the true expectation over all possible trajectories, we estimate it from a finite batch of trajectories collected from the simulator. Actions that led to better-than-expected outcomes are made more likely; actions that led to worse-than-expected outcomes are made less likely.

Crucially, this does not require differentiating the simulator. The simulator is used only to sample transitions and rewards. The gradient is taken through the policy's log-probability of the sampled action, $\log \pi_\theta(a_t \mid s_t)$, which is differentiable because the policy is a neural network. In this project, KomaMRI can therefore remain a black-box physics simulator: PPO samples sequence actions from the current policy, evaluates them with the simulator, and then increases the probability of actions with positive advantage while decreasing the probability of actions with negative advantage.

The weighting term above is written as an advantage estimate $\hat{A}_t$ rather than the raw return G_t. This reduces variance and makes the update depend on whether an action was good relative to what the policy already expected in that state.

Value and advantage functions

The value function of a policy is the expected future return from a state:

$$V^\pi(s) = \mathbb{E}_{\tau \sim \pi}[G_t \mid s_t = s]. \tag{2.17}$$

The action-value function is the expected return after taking a particular action and then continuing with the policy:

$$Q^\pi(s, a) = \mathbb{E}_{\tau \sim \pi}[G_t \mid s_t = s, a_t = a]. \tag{2.18}$$

Equivalently, it can be written in one-step form as the expected immediate reward plus the value of the next state:

$$Q^\pi(s_t, a_t) = \mathbb{E}\left[r_t + \gamma V^\pi(s_{t+1}) \mid s_t, a_t\right]. \tag{2.19}$$

The advantage function compares these two quantities:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s). \tag{2.20}$$

An action with positive advantage performed better than the policy's average action at that state;

an action with negative advantage performed worse. This is the quantity used to decide which sampled actions should become more or less probable.

In deep RL we usually do not know V^π, Q^π, or A^π exactly. PPO therefore learns two approximations at the same time:

- an actor, $\pi_\theta(a \mid s)$, which chooses acquisition actions;
- a critic, $V_\phi(s)$, which predicts the expected return from the current state.

The critic provides a baseline for the policy-gradient update. A simple one-step temporal-difference residual is formed from an actual sampled transition. At time t, the actor samples an action $a_t \sim \pi_\theta(\cdot \mid s_t)$; the simulator applies this acquisition action and returns the immediate reward r_t and the next state s_{t+1}. The residual is then

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t). \tag{2.21}$$

Thus $V_\phi(s_t)$ is the critic's prediction before taking the action, while $r_t + \gamma V_\phi(s_{t+1})$ is the one-step target after seeing the simulator's response to the sampled action.

Generalised Advantage Estimation (GAE) combines these residuals over multiple future steps:[22]

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \tag{2.22}$$

where λ controls the bias-variance trade-off. In words, the value network learns what return is expected from a state, and the advantage estimate measures whether the sampled action improved on that expectation. For this project, that means the critic learns to predict future improvement in the quantitative MRI objective, while the actor learns which sequence parameters tend to produce those improvements.

This is still estimating the same conceptual quantity $A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$. PPO first collects a batch of complete episodes or fixed-length rollout fragments, so the rewards along the sampled trajectory are already known when $\hat{A}_t$ is computed. Using the full observed return would give a Monte Carlo estimate of Q^π, but it can have high variance because it depends heavily on one particular noisy trajectory. Using only the one-step residual relies more on the learned critic, which reduces variance but introduces bias if the critic is inaccurate. GAE interpolates between these two estimates: smaller λ trusts the critic more, while larger λ uses more of the sampled future rewards.

Proximal policy optimisation

Vanilla policy-gradient updates can be unstable because a single gradient step can move the policy too far. If the new policy differs greatly from the policy that generated the rollout batch, the sample estimate may no longer describe the behaviour of the updated policy. Trust Region Policy Optimisation (TRPO) addresses this by constraining the KL divergence between the old and new policies, but it requires a more complex second-order optimisation procedure.[23]

The underlying update is an update to the neural-network weights θ. Even if the parameter change looks small in weight space, it can cause a much larger change in the induced probability distribution over actions, $\pi_\theta(\cdot \mid s)$. Since PPO is on-policy, the rollout batch was collected using the old policy. If the update makes the new policy assign very different action probabilities, the batch becomes stale: it reflects what was useful under the old action distribution, not necessarily what is useful under the new one. Large updates can therefore overreact to lucky sampled actions or collapse exploration.

Proximal Policy Optimisation (PPO) keeps the same basic idea – improve the policy while preventing large destructive updates – but uses a simpler clipped objective.[24] For samples collected with the old policy $\pi_{\theta_{\text{old}}}$, define the likelihood ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}. \tag{2.23}$$

If $r_t(\theta) > 1$, the new policy assigns higher probability to the sampled action than the old policy did. If $r_t(\theta) < 1$, it assigns lower probability. PPO maximises

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t, \operatorname{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)\right]. \tag{2.24}$$

For positive-advantage actions, the objective encourages increasing their probability, but only up to the clipping threshold. For negative-advantage actions, it encourages decreasing their probability, again only up to the threshold. The clipping term therefore removes the incentive for the policy to move far away from the sampled policy in a single update.

The full optimisation also trains the value function and usually includes an entropy bonus to preserve exploration:

$$L(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_v \mathbb{E}_t\left[\left(V_\phi(s_t) - \hat{G}_t\right)^2\right] - c_H \mathbb{E}_t\left[\mathcal{H}(\pi_\theta(\cdot \mid s_t))\right]. \tag{2.25}$$

Here c_v and c_H weight the critic loss and entropy bonus. This is the form used by most practical PPO implementations: collect rollouts using the current policy, estimate returns and advantages, update the actor and critic for a few epochs using minibatches, then discard the old rollout data and collect new on-policy data.

Adaptive qMRI sequence design is naturally sequential. The usefulness of a new measurement depends on what has already been acquired and on the current uncertainty in the tissue-parameter estimate. PPO is a practical choice because it supports continuous action spaces, stochastic exploration, and stable on-policy learning. The policy can explore different acquisition timings and flip angles while the clipped objective limits sudden policy collapse.

The main cost is sample acquisition. Each PPO update requires fresh rollouts from the current policy. In this project, one rollout step is not a cheap game transition: it may require constructing an MR sequence, running a Bloch simulation in KomaMRI, reconstructing the image, extracting phantom regions of interest, fitting T_1, and computing the error-based reward. This makes the number of simulator calls the main training bottleneck.

2.3.5 Clinical Implications

This research landscape is the early phases of a transition from human-engineered MRI protocols to a paradigm of ”autonomous radiology.” While the MR-LINAC offers unparalleled soft-tissue visualisation, its clinical utility is currently throttled by the latency of traditional imaging and manual contouring. The integration of Bayesian reinforcement learning [14], end-to-end supervised learning [11], and task-specific autonomous control [25] could provide a technical pathway to overcome these limitations. By treating the MR-LINAC not as a static imaging tool, but as a dynamic, AI-controlled sensor, it becomes possible to collapse the ”imaging-planning-delivery” cycle into a single, synchronised event. This evolution toward an automated, closed-loop system will likely be the catalyst that allows MR-guided radiotherapy to achieve its full potential, providing real-time, biologically adaptive treatments that respond to tumour dynamics.

3

MRISystemPhantom.jl — Digital Twin Library

Contents

3.1 The physical phantom

The QalibreMD NIST/ISMRM System Standard Model 130 (manufactured by Caliber MRI, formerly QalibreMD) is a hardware phantom designed to mimic the geometry of a human head and to provide a wide, calibrated range of quantitative MRI tissue parameters (T1, T2, and proton density). It is the standard reference device for quantitative MRI system validation in clinical and research settings, and its design is described in the *QalibreMD System Standard Model 130 User Manual* (available here).

The phantom consists of a spherical water-filled housing (100 mm internal radius) containing four internal structures at fixed axial positions:

Plate	z (mm)	Contents
T1 array	+56.5	14 $NiCl_2$ spheres (15 mm radius), T1 range ~24 ms – 1.9 s at 3 T
T2 array	+16.5	14 $MnCl_2$ spheres (15 mm radius), T2 range ~87 ms – 2.8 s at 3 T
PD array	-23.5	14 spheres of varying proton density (15 mm radius)
Fiducial grid	distributed	57 small spheres (5 mm radius) on a 40 mm cubic lattice

Each contrast plate arranges its 14 spheres in two concentric rings: 10 on an outer ring of radius 65 mm and 4 on an inner ring of radius 28 mm. This geometry matches the phantom manual photographs but has not been verified against the physical device. The current digital twin models the contrast arrays and fiducial grid, but not the resolution insets or the two slice-profile wedges.

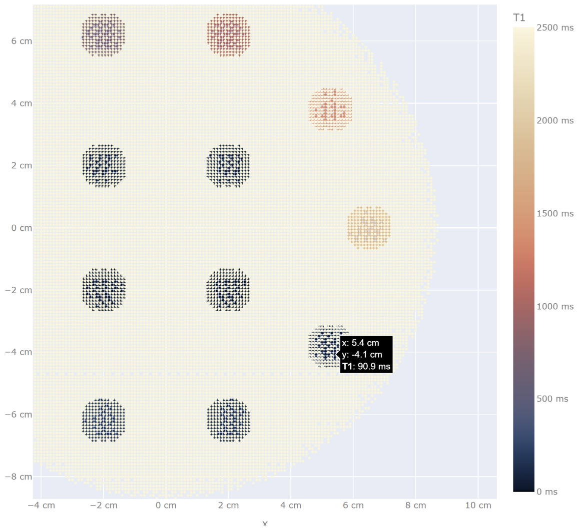


Figure 3.1: **The QalibreMD NIST/ISMRM System Standard Model 130 hardware phantom and its digital T1 array.** The physical phantom photo (left) is reproduced from Caliber MRI. The T1 array (right) is voxelised into a 3 mm-thick KomaMRI phantom: the yellow spots are background-water voxels, while the spheres contain differing T1 values.

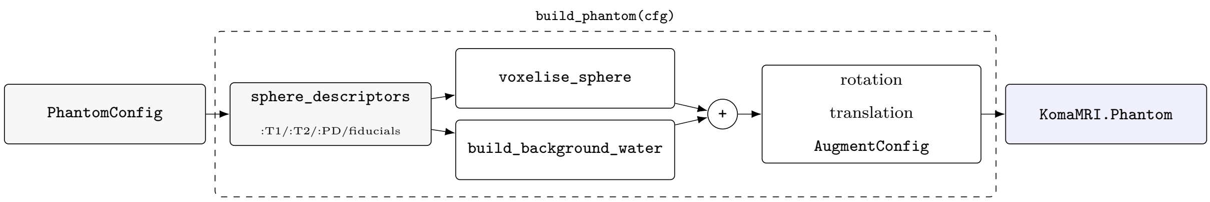
The relaxation values are field-strength-dependent, so separate reference values are provided for 1.5 T and 3 T. The user manual also distinguishes between legacy and modern serial-number classes; both sets of reference values are included in the library.

3.2 Design goals and architecture

A digital twin of this phantom is needed for two reasons. First, it provides a ground-truth environment for RL training: the agent needs to query a simulator that returns physically meaningful signals and can report the true tissue parameters for reward computation. Second, it must be **flexible**: the RL training loop needs to randomise pose, inject noise, and jitter reference values across episodes without modifying library code. Different experiments also require different phantom sections or resolution.

The library is structured around a single contract: a `PhantomConfig` struct that carries the builder configuration. A single function, `build_phantom`, returns a `KomaMRI.Phantom`, which can then be fed directly to `KomaMRI.simulate`. This design was chosen deliberately over a mutable phantom object or a collection of keyword arguments as the config is serialisable, diffable, and reproducible (the RNG seed is embedded), and the builder has no hidden state. This is useful for comparing different RL runs and also simplifies the Python-Julia boundary used by the training code.

The pipeline from `PhantomConfig` to `KomaMRI.simulate()` is:



The two-stage design — descriptors first, voxels second — means all geometry and material decisions happen before any memory is allocated for spin arrays. Descriptors are also individually modifiable and removable: you can simulate only a subset of the 14 spheres per plate, with the water housing automatically filling the vacated volume. Because each descriptor carries its physical centre coordinate, `sphere_descriptor_pixels` can convert those centres directly to pixel indices in the reconstructed image — using the same DC-at-centre convention as `kspace_to_image` — simplifying ROI extraction after reconstruction.

3.3 Geometry

The geometry implementation is judged by whether it preserves the reconstructed sphere measurements, not by whether every background-water spin is represented exactly.

3.3.1 Sphere voxelisation

Each sphere is voxelised on a cubic grid at spacing `delta_x = voxel_size_mm * 1e-3` metres. A spin is included if its grid point falls within the sphere radius:

$$(x_i - c_x)^2 + (y_i - c_y)^2 + (z_i - c_z)^2 \leq r^2$$

The grid is constructed by iterating over the bounding box `[c - r, c + r]` on each axis. For a typical 15 mm radius sphere at 2 mm voxels this produces around 500 spins per sphere. Across all three contrast plates (42 spheres), the combined spin count at 2 mm voxels is ~9,100, occupying ~570 KB (8 Float64 fields per spin: $x, y, z, \rho, T_1, T_2, T_2^*, \Delta\omega$), small enough that the full set fits comfortably in memory and the per-step simulate call is fast.

3.3.2 Slicing

For 2D imaging experiments, only spins within a thin axial slab need to be simulated, reducing the spin count, and therefore simulation time, substantially. A `Slab` is defined by a centre point **c**, a unit normal $\hat{n}$, and a thickness t. A spin at position **p** is included if its signed distance from the midplane satisfies:

$$|(\mathbf{p} - \mathbf{c}) \cdot \hat{n}| \leq t/2$$

The normal defaults to $\hat{z}$ but any orientation is supported. A 0.1 μm tolerance is applied at the boundary to avoid floating-point rejections of on-edge voxels. At 1 mm voxels, a 5 mm axial slab centred on the T1 plate retains ~33,000 of the full phantom's ~4.2 million spins, a 128× **reduction**. This matches the excited volume of a slice-selective RF pulse without needing to encode one explicitly.

3.3.3 Pose transforms

`apply_transform!` rotates and translates all spin positions by the Euler XYZ rotation and translation specified in `PhantomConfig.rotation` and `translation_mm`. The rotation matrix is built from three successive Rx, Ry, Rz multiplications. The transform is applied after voxelisation and before augmentation noise so that the sphere geometry is exact before any stochastic perturbation. This was designed to make it easy to reset the phantom and reduce overfitting.

3.3.4 Background water

The housing is voxelised as a 100 mm-radius sphere with `water_voxel_size_mm` spacing (defaulting to `voxel_size_mm`), with all contrast and fiducial sphere volumes cut out by the negated form of the squared-distance test used for voxelisation. The water housing is by far the largest spin population, yet it is not a quantity the phantom exists to measure, so it is the natural place to trade fidelity for simulation speed. We do this by increasing `water_voxel_size_mm` to coarsen the grid, thus reducing the spin count. We then reweight each spin's proton density by `(water_dx / sphere_dx)³`. This conserves the total water magnetisation, so the bulk water signal amplitude is correct regardless of the coarsening factor.

This coarsening must also handle anisotropic resolution as slices are typically thick (3–8 mm) relative to the in-plane resolution (1–2 mm). So, when a slice is selected the housing is projected onto a grid of stacked plane-sheets (`water_throughplane_voxel_size_mm` apart). Again, these are re-weighted to conserve total magnetisation.

The figure below quantifies the fidelity–cost trade for an IR-SE-2D acquisition.

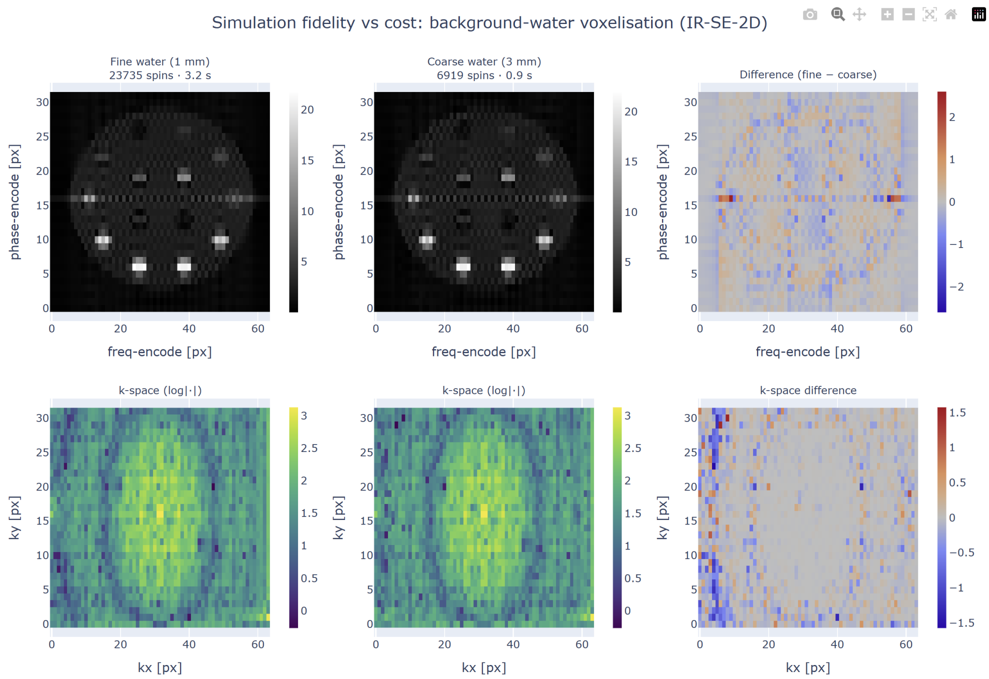


Figure 3.2: **Fidelity–cost trade-off of background-water voxelisation.** The same IR-SE-2D acquisition (TI/TE/TR = 400/80/1500 ms, 32 × 64, FOV 0.2 m) is simulated through two phantoms differing *only* in background-water discretisation. **Top row:** magnitude images; **bottom row:** log-magnitude k-space; **right column:** fine - coarse difference. The figure and numbers are reproduced by `examples/compare_water_coarseness.jl`.

Conserving the bulk water signal is necessary but not sufficient. The k-space DC term changes by only 0.35%, but the phantom is used to measure the sphere signals. The coarse 3 mm grid uses **6,919 vs 23,735 spins** (3.4× fewer) and is ~3.4× **faster to simulate** (0.9 ± 0.1 s vs 3.2 ± 0.6 s, mean ± std over 20 runs). Using the `sphere_descriptor_pixels` mapping, the single centre-pixel coarse-vs-fine NRMSE at the 32×64 matrix is **3.7%**.

The larger single-pixel error reflects where the coarse grid differs from the fine one. Reweighting preserves the amount of water, but not the exact water/sphere boundary: the 3 mm water grid stair-steps the sphere cut-outs, changing high-spatial-frequency edge content while leaving low-frequency k-space essentially unchanged. The boundary error is on the scale of the reconstructed pixels here: for a 200 mm FOV and 32×64 matrix, the spacing is 6.25 mm in phase encode and 3.125 mm in frequency encode. After band-limited reconstruction, this boundary mismatch appears as a slightly different Gibbs-ringing pattern around the spheres.

Most of this error is concentrated in the near-null spheres at this inversion time, especially T1-spheres 13-14, whose signals move by about 9%, while the bright spheres change by $\leq 0.7\%$. This is expected because the ringing leakage is approximately additive, set mostly by the surrounding bright water.

If the sphere signal is measured as a small central ROI rather than a single point sample, the error falls to **1.5%** for a 3×3 ROI. This is the more relevant operational metric for an ROI-based phantom readout; the single-pixel value remains useful as a worst-case sensitivity measure because it exposes pixel-phase effects, Gibbs oscillation, and sub-pixel centre rounding.

The same pattern strengthens at 64×64: with unchanged spin counts and DC term, the errors rise to **7.6%** for a single centre pixel and **2.3%** for the 3×3 ROI, consistent with a wider k-space window admitting more of the edge mismatch.

Because the discrepancy is carried by truncation side-lobes, it can be suppressed at reconstruction. The library provides a 2-D Hamming window through `hamming_window_2d` and `kspace_to_image(...; hamming=true)`. As derived in the background chapter, this reduces the PSF side-lobes from -13 dB to -43 dB, at the cost of a wider main lobe.

Reconstructing both grids with the Hamming window reduces the single-pixel coarse-vs-fine NRMSE by four-fold, from **3.7% to 0.9%**. This confirms that the single-pixel discrepancy was dominated by truncation ringing. The 3×3-ROI error improves more modestly, from **1.5% to 1.1%**, because ROI averaging already cancels much of the oscillatory side-lobe structure. Apodisation and spatial averaging therefore suppress largely the same error component, rather than providing independent gains. The remaining $\approx 1.1\%$ is the residual coarse-grid discrepancy under this ROI-based measurement protocol.

The Hamming window down-weights outer k-space, visible as the darkened periphery of the centre k-space panel in Figure 3.3. This widens the PSF main lobe and slightly blurs the spheres, but suppresses ringing side-lobes. The difference is localised to sphere edges and high-frequency k-space, consistent with suppression of truncation ringing rather than a change in the smooth low-frequency signal. The window

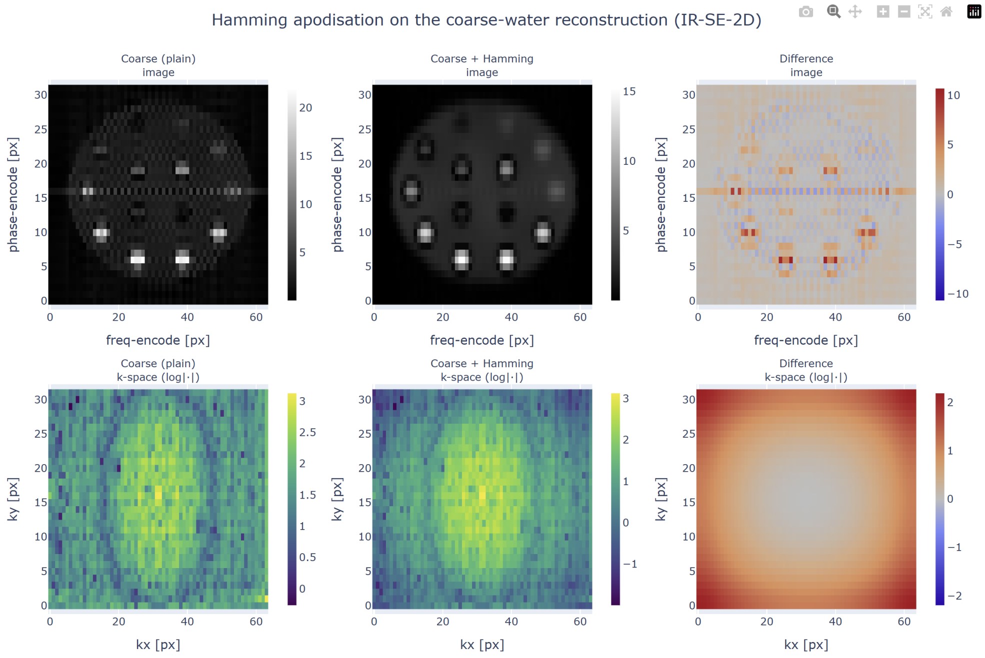


Figure 3.3: **Effect of Hamming apodisation on one reconstruction.** A coarse-water phantom reconstructed without a Hamming window (left), with a 2-D Hamming window (centre), and their difference (right). Produced by `examples/hamming_apodisation.jl`.

is a reconstruction-side element-wise multiplication and costs only a fraction of a millisecond, so it does not affect the roughly 3× simulation speed-up from the coarse grid. Its cost is resolution, through the wider PSF main lobe.

A natural future improvement is a boundary-aware water grid: keep the smooth water bulk coarse, but voxelise a thin shell around the housing surface and sphere cut-outs at fine resolution so that coarsening no longer changes the boundary geometry.

3.4 Material tables

The relaxation values are read from four module-level constants defined from the phantom manual:

Constant	Spheres	Source
`T1_ARRAY[:T3] / [:T15]`	14 T1-plate spheres	$NiCl_2$ recipe, manual table, Serial Number $\geq$ 0042
`T1_ARRAY_LEGACY[:T3] / [:T15]`	same spheres	manual table, SN pre-0042 (different values)
`T2_OF_T1_ARRAY`	T2 companions for T1 plate	same manual table
`T2_ARRAY[:T3] / [:T15]`	14 T2-plate spheres	$MnCl_2$ recipe
`T1_OF_T2_ARRAY`	T1 companions for T2 plate	same manual table

Constant	Spheres	Source
`PD_FRACTIONS`	14 PD-plate spheres	proton density fractions
`BACKGROUND_WATER`	housing water	distilled water at 20°C

The T1 range at 3 T spans two decades: 23 ms to 1.84 s for the $NiCl_2$ plate, covering the full range of biological tissue T1 values from white matter to cerebrospinal fluid. The T2 range spans ~87 ms to 2.76 s. These wide ranges are why the phantom is used for system validation and make it useful for RL training: to ensure the learned policies can produce reliable estimates across the full physiological range.

The `serial_number_class` field switches the T1 table between current and legacy recipes. All reference values are currently fixed at 20°C; the manual includes temperature-correction graphs, but these are not yet implemented.

Accuracy caveats. Two entries are approximate: PD-plate T1/T2 values (`pd_t1`, `pd_t2`) use bulk water regardless of proton-density fraction (the manual does not tabulate per-sphere relaxation for the PD plate), and fiducial relaxation values (`FIDUCIAL_PROPS`) are generic $CuSO_4$ placeholders not verified against any calibration record. Neither affects the T1- or T2-plate experiments, but both should be replaced before any simulation-to-real comparison involving the PD plate or fiducial grid.

3.5 Augmentation

`AugmentConfig` enables domain randomisation for RL training. All fields default to zero. When non-zero values are provided, `apply_per_spin_noise!` draws from the relevant distributions using the config's `rng_seed`:

Field	Effect
`T1_sigma_rel`	Multiplies each sphere's T1 by `exp(sigma*z)` where `z ~ N(0,1)`, drawn once per sphere
`T2_sigma_rel`	Same for T2
`PD_sigma_abs`	Adds `N(0, sigma^2)` to each spin's proton density
`position_sigma_mm`	Adds `N(0, sigma^2)` to each spin's (x, y, z) position independently
`B0_sigma_Hz`	Adds `N(0, sigma^2)` per-spin off-resonance, stored in Δw (rad/s after $\times 2\pi$)

Field	Effect
`drop_sphere_p`	Drops each sphere independently with Bernoulli probability p before voxelisation

The T1/T2 jitter is applied at the sphere level (one draw per sphere, not per spin) to simulate manufacturing variability in the doping concentration. Of the fields above, only `T1_sigma_rel` and `T2_sigma_rel` were used in the experiments of this work; the remaining fields are implemented for completeness but were not exercised.

The `rng_seed` is the only source of stochasticity: fixing it makes `build_phantom` deterministic, so the same configuration always produces the same phantom. Fixing the seed reproduces the exact same phantom, making evaluation runs reproducible. Incrementing it each training episode draws a statistically independent phantom, preventing the agent from memorising a fixed configuration.

3.6 Shipped sequences and fitting models

The background chapter explains what the main MRI sequence families are. This section instead documents what `MRISystemPhantom.jl` ships, why those routines are needed by the experiments, and which assumptions they encode. All sequence builders return standard `KomaMRI.Sequence` objects, so they can be passed directly to `KomaMRI.simulate`. The fitting routines are pure Julia and are shared by the conventional baselines and RL environments, so reported gains are measured against the same estimators.

Single-spin sequence helpers

- `ir_sequence(TI)`: 180° inversion, delay, 90° excitation and ADC. Used for simple T1 checks and the conventional T1 helper; after full recovery the first ADC sample follows $|1 - 2e^{-\mathrm{TI}/T_1}|$.
- `se_sequence(TE)`: 90°–180° spin echo for the T2 baseline. The echo magnitude follows $S_0 e^{-\mathrm{TE}/T_2}$.
- `mse_sequence(ESP, n_echoes)`: CPMG / multi-echo spin echo. One excitation is followed by a train of 180° pulses; echo k occurs at $k\,\mathrm{ESP}$, so a whole T2 decay is sampled in one TR.

2D Cartesian sequence builders

- `ir_se_2d_sequence(TI, TE, TR; ...)`: main spatial T1 acquisition for E2. It is parameterised by α_{exc}, `FOV`, `Nfe` and `Npe`; it acquires one k-space row per shot, so fits pass `Npe` to account for the transient ramp from thermal equilibrium.

- `ir_tse_2d_sequence(TI, esp, TR; ...)`: turbo spin-echo variant with echo-train length `etl`. It is faster by approximately `etl` and exposes T2 information when TE varies across echoes.
- `se_2d_sequence(TE, TR; ...)`: 2D spin echo without inversion preparation, used for spatial T2 mapping by sweeping TE.
- `gre_2d_sequence(TE, TR; ...)`: fast gradient-echo reference with flip angle α. It has no 180° refocus, so it is T2*-weighted; the implementation includes gradient spoiling only, not full RF-spoiled FLASH phase cycling.

Analytical signal models

- `generalized_ir_signal`: non-spatial IR signal for arbitrary inversion angle. π gives inversion recovery and $\pi/2$ gives saturation recovery.
- `mse_signal`: analytical CPMG echo train, $e^{-k\,\mathrm{ESP}/T_2}$. Used as a fast T2 oracle in tests.

Fitters and forward models

- `fit_t2_se(TEs, mags)`: log-linear estimator for $|S| = S_0 e^{-\mathrm{TE}/T_2}$.
- `fit_t1_ir(TIs, mags)`: conventional long-TR magnitude IR estimator, fitting $|A - Be^{-\mathrm{TI}/T_1}|$.
- `fit_t1_generalized_ir`: main RL estimator. It accepts per-block inversion angles, optional `TRs`, excitation angles and `Npe`; `Npe = nothing` selects the steady-state model and integer `Npe` selects the transient mean over the acquired phase-encode shots.
- `fit_t1_t2_generalized_ir`: joint T1/T2 grid fit for IR-TSE-style data. T2 is identifiable only when TE varies, because a constant TE factor can be absorbed into the fitted amplitude.
- `steady_state_mz_at_excite`, `transient_mz_at_excite_npe` and `transient_mz_per_shot`: shared longitudinal forward models used by the fitters, tests and CR-style baselines.

All image-producing sequences use the same reconstruction path: `raw_to_kspace(raw, Npe, Nfe)` followed by `kspace_to_image(ksp)`. Sphere-centre ROI extraction uses the same coordinate convention through `sphere_descriptor_pixels`, avoiding a separate hand-coded image-space mapping in the experiments. The heavier fit algebra is kept out of the main implementation chapter; Appendix A summarises the derivations that matter for interpreting the estimators.

3.7 KomaMRI integration

`MRISystemPhantom.jl` is built entirely on top of `KomaMRI.jl` and returns standard `KomaMRI.Phantom` objects. The whole phantom is assembled using a single operation: the concatenation operator `+` on

`Phantom`. Each voxelised sphere becomes a small `Phantom`, the spheres of each T1, T2 and PD plate are concatenated into that plate, the three plates are concatenated into a sphere stack, and finally the water background is concatenated on top. The result is a single array of spins with heterogeneous relaxation values, which is exactly what `KomaMRI.simulate` expects.

Because the output is an ordinary `KomaMRI.Phantom`, existing KomaMRI features work out of the box, such as the interactive plotting tools which are used extensively in the example scripts.

The matching scanner is obtained from `scanner_for_field(cfg)`, which returns a `Scanner` with B0 set to 3.0 T for `:T3` or 1.5 T for `:T15`. Pairing a phantom with a `Scanner` of a different B0 silently produces the wrong relaxation values - a mistake the author made in practice - so this helper exists to make the coupling explicit.

3.8 Library availability

`MRISystemPhantom.jl` is developed as a standalone open-source Julia package, with source code hosted on GitHub and API documentation hosted on GitHub Pages. The package includes a complete test suite (~373 tests) and documentation pages covering phantom construction, sequence design, parameter fitting, and SNR diagnostics.

An independent, public library has several benefits:

1. It provides a complete, tested forward pipeline — phantom construction, sequence generation, simulation, reconstruction, ROI extraction, and parameter fitting — rather than a set of experiment-specific scripts.
2. It makes the methodology reproducible and extensible for other groups working on quantitative MRI system validation or simulation-based sequence optimisation.
3. It provides a reusable digital twin of a widely deployed calibration device, so simulated methods can be checked against known T1/T2 reference values before scanner deployment.
4. It integrates directly with the Julia/KomaMRI ecosystem by returning standard `KomaMRI.Phantom` and `Sequence` objects.
5. Its example scripts provide runnable, educational references for phantom construction, sequence design, parameter fitting, SNR calibration, and the fidelity checks reported here.

3.8.1 Example scripts

The package ships a set of runnable, self-contained demonstration scripts in `examples/`. Each exercises one slice of the public API and writes interactive Plotly HTML into `src/assets/`.

- `plot_phantom.jl`: renders the T1/T2/PD maps and slice cuts of the built phantom.
- `plot_fidelity_phantoms.jl`: visualises the water-coarsening fidelity levels.
- `t1_mapping.jl`: runs a full IR-SE acquisition, reconstructs the image, extracts per-sphere ROIs and fits T1.
- `conventional_baseline.jl`: runs the conventional fixed-sequence baseline (IR/SE sweep plus fit for all 28 T1 & T2 contrast spheres).
- `snr_calibration.jl`: performs the NEMA MS-1 dual-acquisition SNR measurement.
- `compare_water_coarseness.jl` and `hamming_apodisation.jl`: produce the fidelity-vs-cost and apodisation figures (§2.3) along with the numbers quoted there.
- `plot_seqs.jl`: writes one interactive RF/gradient/ADC waveform plot per sequence into `src/assets/sequences/`. Sequences included: IR-SE (with and without crusher & TR-spoiler variant), SE for T2 mapping, IR-TSE echo-train, and spoiled GRE.

Because the output is a standard `KomaMRI.Phantom/Sequence`, these scripts rely only on KomaMRI's own interactive plotting, with no bespoke visualisation code in the library.

4
Validating KomaMRI

KomaMRI is the simulation environment that all RL agents in this project interact with, so simulator validation is a prerequisite for every result in this project. Although KomaMRI is a peer-reviewed, open-source, and actively maintained MRI simulation framework [10], the validation experiments in this project found two floating-point time-discretisation failures. Both were triggered by long cumulative sequence time. The failures were reduced to minimal reproducers and fixes were contributed upstream: issue #779 / PR #780, now merged, and issue #788 / PR #789, open at the time of writing.

This work is part of A1, the validated digital twin and simulator pipeline, and directly addresses C1: simulator validation for qMRI. For A1, the key contribution is the validation of the simulation pipeline rather than the upstream reports alone. The two failures were reduced to minimal reproducers, traced to specific floating-point timing mechanisms, fixed upstream, and then verified by recovering the phantom's known T_1 values in the full qMRI pipeline.

4.1 KomaMRI simulation model

KomaMRI simulates the magnetization of each spin of a `Phantom` for variable magnetic fields given by a `Sequence` [10]. All `Phantom` objects used in this project were produced by `MRISystemPhantom.jl`. Each `Phantom` is a cloud of spin isochromats: groups of nearby nuclei that are assumed to share a single position (x, y, z) and one set of tissue parameters $(T_1, T_2, T_2^*, \rho, \Delta\omega)$. Here ρ is proton density and $\Delta\omega$ is off-resonance: the offset of that isochromat's Larmor frequency from the nominal scanner frequency. Each isochromat is therefore represented by a magnetisation vector (M_x, M_y, M_z) evolving according to the Bloch equations.

The user-facing interface is:

```
raw = simulate(phantom, sequence, scanner; sim_params)
```

Here `scanner` defines the scanner hardware used to play the sequence. In these experiments it is an idealised hardware model: eddy currents and field inhomogeneities are not modelled. This means RF pulses are treated as exact, although in reality a 180° refocus pulse can vary by a couple of degrees across the excited volume.

A `Sequence` is made up of RF pulses, which rotate the magnetisation vector, as well as gradients, delays, and ADC readouts. Internally, `simulate` represents the continuous sequence on a discretised time grid. Gradient events use a time step Δt, while RF pulses are sampled more finely using Δt_{rf}. These time points are then split into blocks. In excitation blocks, where RF is active, KomaMRI integrates the full magnetisation rotation. In precession blocks, where RF is off, the update reduces to relaxation and phase accrual, which can be computed analytically with exponentials and is cheaper to evaluate. At each ADC sample, the simulator sums the transverse magnetisation over all spins to form the complex received signal $\mathrm{sig}[t]$.

The key computational assumption is that spins evolve independently and only interact through the final signal sum. This is a standard MRI simulation approximation and is less restrictive than the hardware idealisations above. It is also what makes KomaMRI practical for this project: spins can be partitioned across CPU threads or GPU kernels, then summed only at the receiver. This parallelism motivated the use of KomaMRI and helps overcome C2, the cost of simulation. KomaMRI also models bulk T_2^* decay rather than microscopic spin-spin interaction. Overall, these are standard MRI simulator assumptions that model the relevant physics to an acceptable error, so learned policies should plausibly generalise to real hardware.

4.2 Initial validation failure

The first validation test was deliberately simple. With no added noise, the test simulated an inversion-recovery sweep across the 14 T_1 spheres and fitted the standard monoexponential recovery model:

$$M_z(\mathrm{TI}) = M_0\left(1 - 2e^{-\mathrm{TI}/T_1}\right).$$

In a noise-free digital phantom, this should be almost exact. Instead, the fitted T_1 values had a mean absolute percentage error (MAPE) of 39.4%, with some spheres close to 100% error. The recovery points

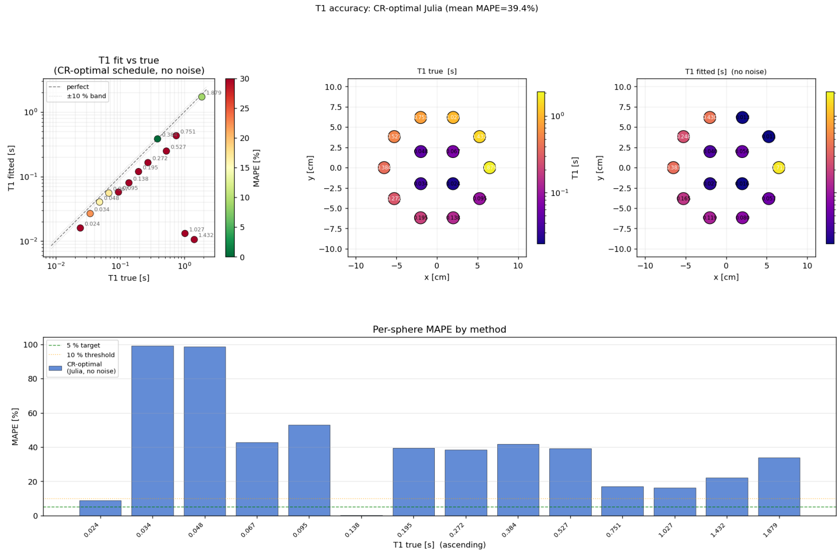


Figure 4.1: Noise-free T_1 recovery before the KomaMRI fixes. The fitted values should lie on the diagonal. Instead the mean MAPE is 39.4%, with only two out of the 14 spheres in the ±10% band.

did not lie on a monoexponential curve. Either the simulator, sequence, reconstruction, or fitting pipeline was wrong, or there was relevant MRI physics not captured by the simple recovery model.

The first step was to rule out plausible MRI explanations. Imperfect transverse spoiling was the most likely candidate: residual M_{xy} can survive one shot, be refocused by later RF pulses, and contaminate the next echo. The standard way to suppress this is to add spoiler gradients, typically around the end of the shot and around refocusing pulses, to dephase any remaining transverse magnetisation. However, when implemented this did not remove the error; it changed mean MAPE from 39.4% to 44.0%. Increasing TR also made the fit worse, even though longer recovery time should reduce both residual transverse coherence and incomplete longitudinal recovery. This was the first strong clue that the failure was driven by total simulated time, not by the physical state at the start of each shot.

Reconstruction explanations were also tested. Hamming-window filtering and ROI masking did not remove the error, making ordinary Gibbs ringing unlikely. A phase-sensitive reconstruction was also checked to ensure the magnitude image was not hiding a sign or phase convention error. These tests changed details of the images, but not the failure mode.

The decisive diagnostic was to inspect k-space directly. Rather than showing random fitting error, the measured k-space collapsed after a fixed amount of simulated time, even though the acquisition should have remained symmetric. This shifted the investigation away from missing MRI physics and toward the simulator's handling of long absolute times.

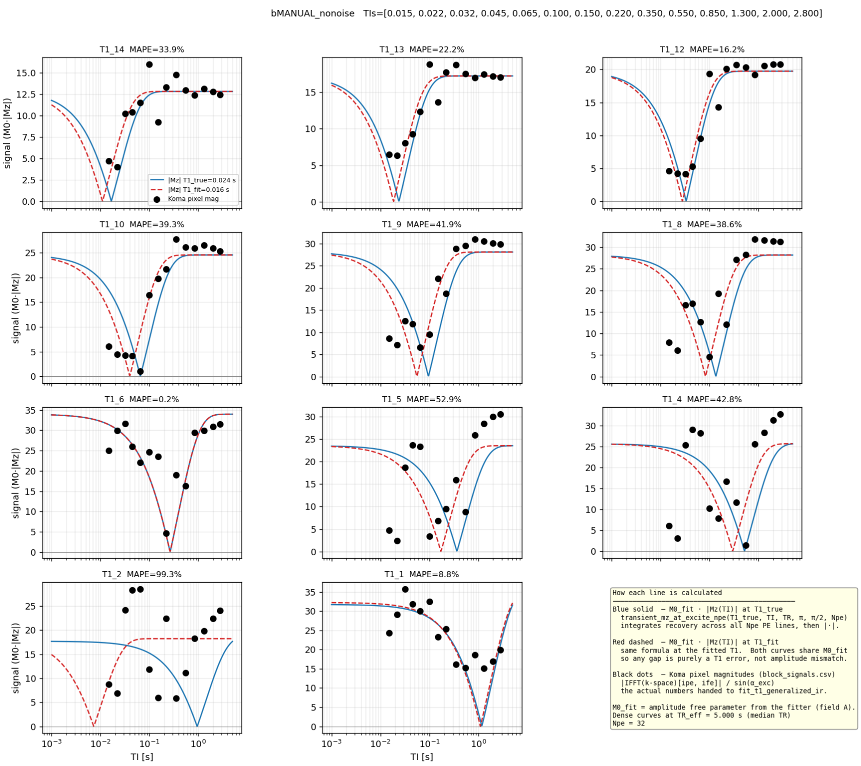


Figure 4.2: Recovery points before the KomaMRI fixes. The simulated samples do not lie on the best-fit monoexponential curves. The deviation is structured, not random, so it points to a repeatable simulation or analysis error.

4.3 RF edge marker collapse

The first minimal reproducer was `koma_bug_minimal.jl`. It removed the phantom geometry and reconstruction and repeated the same simple shot at increasing absolute simulation times. The expected result was a constant per-shot signal. Instead, sharp jumps appeared once cumulative simulated time reached roughly 70–150 s. For example, with TR = 15 s the signal jumped at shot 6 (~90 s) by 21%; with TR = 8 s it jumped at shot 9 (~72 s) by 19%; and with TR = 30 s it jumped at shot 4 (~120 s) by 21%. This showed that the threshold was time-driven, not shot-count-driven, and raised the next question: was KomaMRI discretising the same RF pulse differently at large absolute time?

The RF-edge investigation revealed that the first failure was caused by the way KomaMRI represents sharp RF edges on its simulation time grid. Tracing the RF pulse discretisation led to `points_from_key_times`, and then to the edge-handling logic inside `get_variable_times`. In `get_variable_times`, KomaMRI creates a small set of key times around each RF pulse:

```
points_from_key_times(sort([t1, t1 + MIN_RISE_TIME, tc,
                                    t2 - MIN_RISE_TIME, t2]); dt=Δt_rf)
```

where `t1` and `t2` are the start and end of the pulse, `tc` is the centre. KomaMRI encodes the

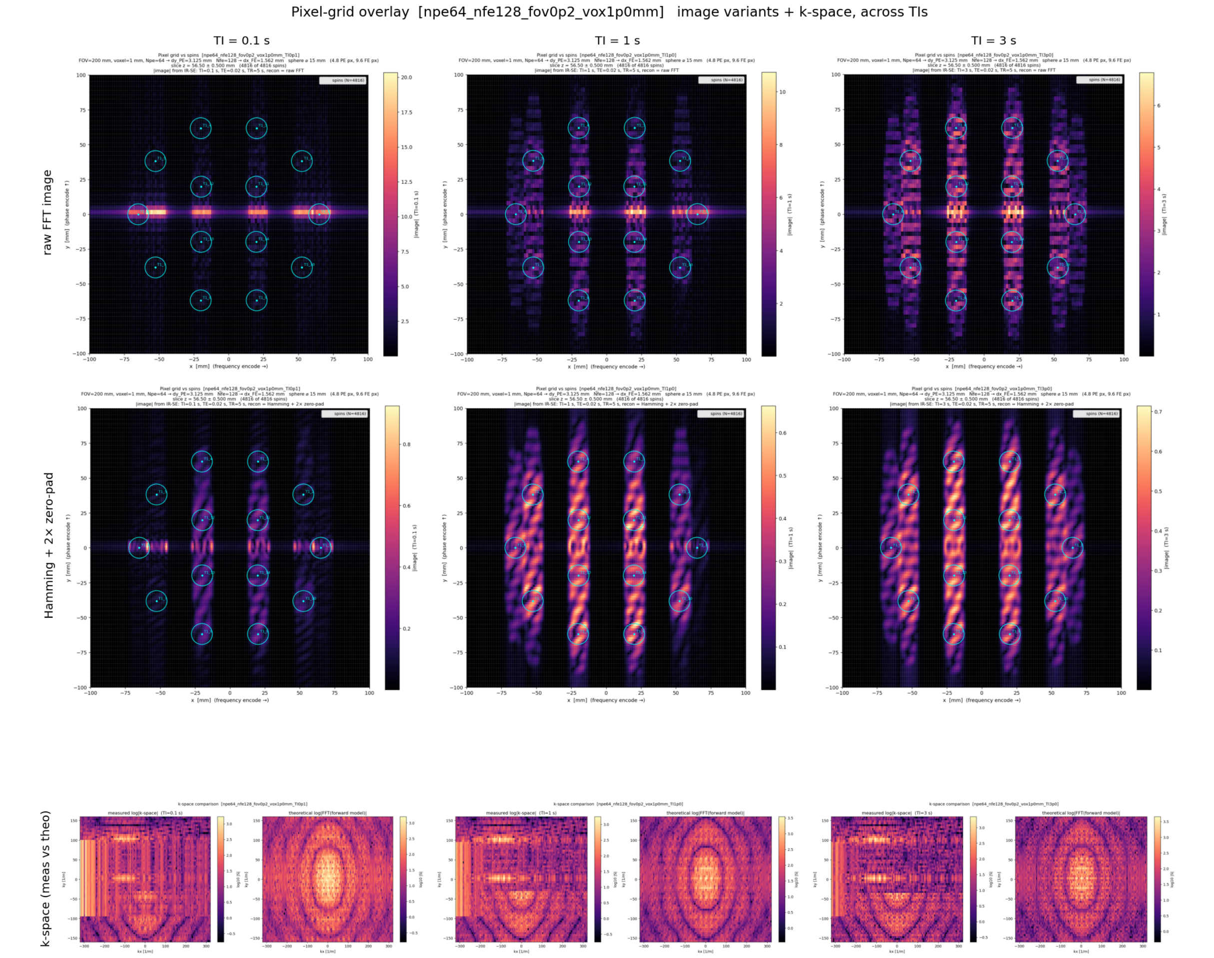


Figure 4.3: Image-space symptom of the timing failure. The figure shows an IR-SE acquisition at three inversion times (TI = 0.1, 1.0, and 3.0 s). The first row shows the reconstructed image, the second row applies a Hamming window, and the bottom row compares theoretical and measured k-space. Because the phase-encode direction (k_y) is acquired shot by shot, moving along k_y corresponds to advancing in simulated time. The early phase-encode lines reconstruct correctly, but later lines collapse into horizontal streaks after a fixed cumulative time. This is too large and structured to be ordinary ringing, and is not removed by the Hamming window. Generated by `pixel_grid_overlay.jl`.

sharp RF edge using a fixed offset $\epsilon = 10^{-14}$ s, implemented as `MIN_RISE_TIME`. The pair `(t1, t1 + MIN_RISE_TIME)` is used to encode the rising edge of the RF pulse. At `t1`, interpolation should return zero RF amplitude; immediately after the edge, at `t1`$+\epsilon$, it should return the full RF amplitude. This creates a near-discontinuous step while still letting KomaMRI store the RF waveform using a small number of key points, with interpolation filling in the values between them.

The problem is that ϵ is a fixed absolute time, but Float64 precision is relative. The spacing between t and the next representable Float64 number, calculated by `eps(t)` in Julia, grows with the magnitude of t: $\mathrm{eps}(t) \approx t \cdot 2^{-52}$. Once $\mathrm{eps}(t)/2 > \epsilon$, adding ϵ no longer produces a distinct Float64 value. In practice this happens at about $t \geq 128$ s. Beyond this point, `t1 + MIN_RISE_TIME == t1` bit-for-bit. The two key times that should define the rising edge therefore collapse onto the same value. When the time vector is sorted and deduplicated, one of them is removed.

The isolated test `koma_bug_isolate.jl` backed this up by asking whether the same 1 ms RF pulse is converted into the same time points at different absolute times:

t_0 [s]	`nraw`	`nunique`	eps(t_0)
0.0	24	23	5.0×10^{-324}
3.0	24	23	4.44×10^{-16}
67.0	24	23	1.42×10^{-14}
99.0	24	23	1.42×10^{-14}
200.0	26	21	2.84×10^{-14}
1000.0	26	21	1.14×10^{-13}

The table shows the collapse directly: early pulses retain 23 unique time points, but at $t_0 = 200$ s this falls to 21 as eps(t_0)/2 exceeds ϵ. This confirms that the RF edge markers have collapsed before simulation.

The collapsed points are not redundant: they represent the two sides of the RF step. At `t1`, RF is zero; at `t1`+ϵ, RF has reached the pulse amplitude. Deduplicating them removes the edge, so the Bloch integrator applies the wrong RF value for one full Δt_{rf} sample. For KomaMRI's default $\Delta t_{\mathrm{rf}} = 5 \times 10^{-5}$ s and $T_{\mathrm{RF}} = 1$ ms, one missing sample is 5% of the RF area. The ϵ-scale timing collapse therefore becomes a 5% flip-angle error, producing a 20–25% signal jump and later a biased T_1 fit.

The fix in PR #780 made the edge markers adaptive:

```
next_time(t) = max(t + MIN_RISE_TIME, nextfloat(t))
prev_time(t) = min(t - MIN_RISE_TIME, prevfloat(t))
```

`nextfloat` and `prevfloat` step to the adjacent representable Float64 value, so the marker remains distinct even at large absolute time. At small times the original spacing is preserved; at large times the marker becomes one floating point step away rather than collapsing. This fixed the first failure and was merged into KomaMRI.

4.4 Closing knot collapse after time rebasing

After PR #780 the direct checks all passed: the isolated RF-edge script was stable to 1000 s, the minimal multi-shot reproducer no longer drifted over the tested shots, and the new RF-area regression test showed that the discretised pulse area matched the expected trapezium area at large absolute time. The original failure mechanism had genuinely been fixed.

However, the full no-noise T_1 validation was still inaccurate. Further investigation revealed another fixed-ϵ collapse, but at a different point in the time pipeline. KomaMRI also separates a block's closing knot, or block edge, using the same fixed offset ϵ in block-relative time. This is initially safe because the local block times are small (e.g. 1 ms RF pulses). Later, however, `get_variable_times` and `get_samples` rebase these local times to absolute sequence time using `T0 .+ t`, where `T0` is the absolute start time of the block. This creates the same floating-point problem as before: at large values of `T0`, the ϵ gap is below Float64 precision. The closing knot rounds onto the block-end knot, the two times become bit-identical, and a later `unique!` step removes one of them. The marker therefore disappears much later in the code than where it was first created, which made this second failure harder to track down.

The reproducer showed that the remaining timing error still mattered. It used one spin and a gradient-free repeated inversion-recovery shot:

```
180 deg inversion -> TI -> 90 deg excitation -> one ADC sample -> TR delay
```

Every shot is physically identical. At TR = 15 s, all 24 shots should return the same signal. Instead, the first 17 shots were stable, shot 18 jumped by 52%, and later shots settled to a new wrong plateau:

```
shot  1..17  (sim_time <= 255 s): |signal| = 0.476267 stable
shot 18      (sim_time  = 270 s): |signal| = 0.722479 +52%
shot 19..24  (sim_time >= 285 s): |signal| = 0.677159 new wrong plateau
```

The fix in PR #789 re-separates the closing knot after rebasing, when the correct absolute-time spacing is known. In implementation terms, the closing knot is moved to at most $t_{\text{end}} - \max(\epsilon, \text{eps}(t_{\text{end}}))$. The gap is adaptive: it remains ϵ at small times, but widens when Float64 spacing becomes larger. The re-separation is also idempotent: if the closing knot is already distinct, it is left alone. The fix is applied at each rebasing site: both RF sampling paths in `get_samples`, and the gradient timing path in `get_variable_times`. The latter also re-applies an adaptive `_strictly_increasing_knots!` check so gradient knots remain ordered after rebasing. With this patch, the 24-shot reproducer is flat and the KomaMRI tests still pass.

At the time of writing, issue #779 is merged upstream via PR #780. Issue #788 is fixed in a project fork pinned by this repository, with PR #789 still open.

4.5 Validation after the fixes

With both fixes applied, the original no-noise phantom test behaves as expected. The reconstructed images no longer develop time-driven streaks, the recovery points lie on monoexponential curves, and the fitted T_1 values return to the diagonal. Mean MAPE falls from 39.4% to 0.48%, with a maximum sphere error of 1.21%.

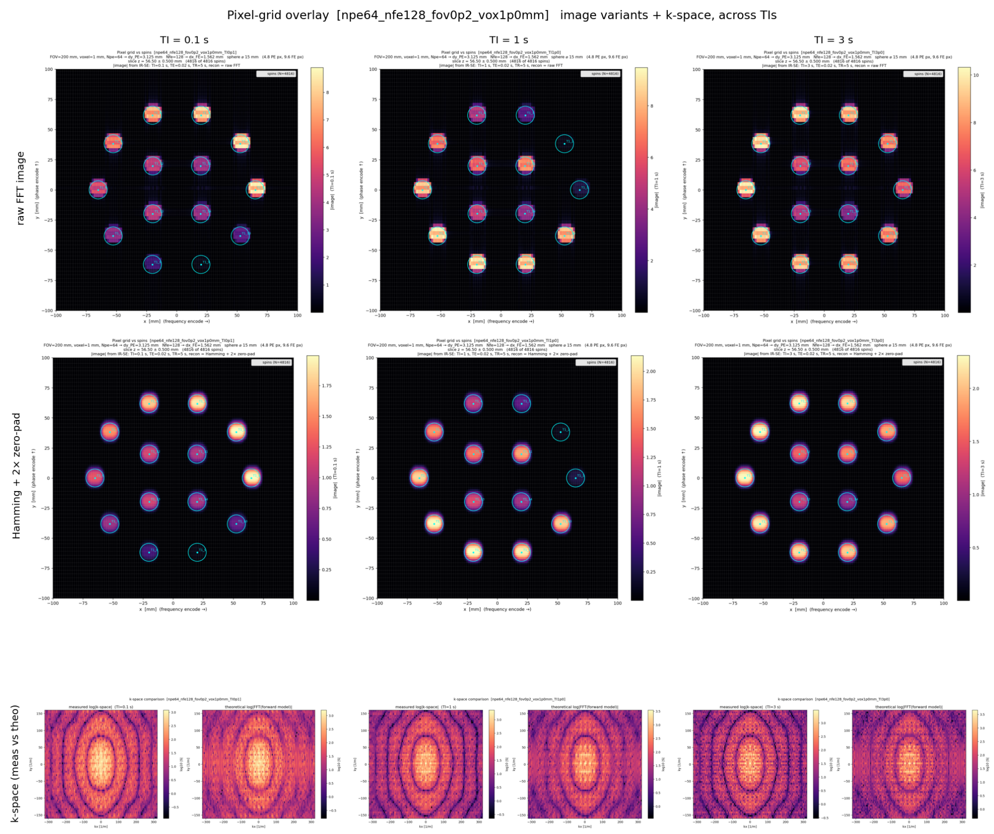


Figure 4.4: Reconstructed images after both fixes. The spheres remain well-localised across the full phase-encode direction. The time-dependent streaking in Figure 4.3 is removed.

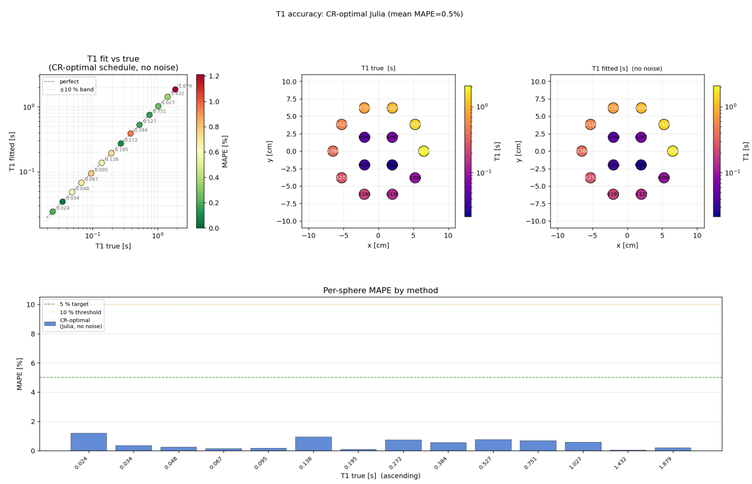


Figure 4.5: Noise-free T_1 recovery after both fixes. Mean MAPE decreases from 39.4% to 0.48%, and all spheres are within 1.21% of their true value. This is the expected accuracy for the noise-free validation case.

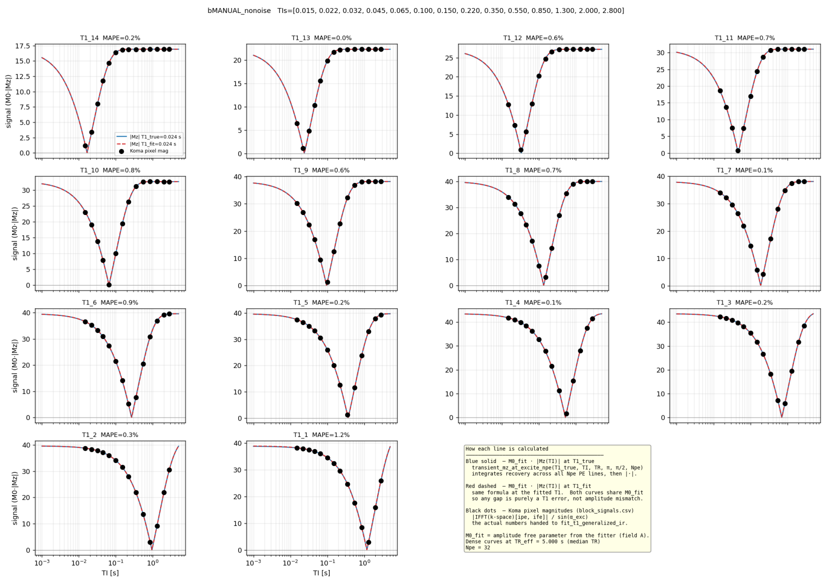


Figure 4.6: Recovery curves after both fixes. The simulated samples now lie on the fitted inversion-recovery curves.

4.6 Evaluation of the fixes

The fixes were evaluated at multiple levels, from isolated timing behaviour to the full qMRI validation task.

Level	Test	Before fix	After fix
Timing representation	RF-edge discretisation at large absolute time	RF edge markers collapse once cumulative time reaches the 70–150 s regime	Stable to 1000 s
Minimal signal reproducer	Identical IR shots, no gradients (`koma_bug_minimal.jl`)	19–21% signal jump once cumulative time reaches 72–120 s	Flat over tested shots
Residual timing reproducer	24 identical IR shots, no gradients (`koma_bug_residual.jl`)	+52% jump at shot 18 (270 s), then wrong plateau	Flat over all 24 shots
Full qMRI validation	No-noise 14-sphere T_1 recovery	39.4% mean MAPE	0.48% mean MAPE (1.2% max MAPE)
Regression safety	KomaMRI test suite	New timing tests fail on unfixed code	Passes after both fixes

Table 4.1: Summary of KomaMRI timing-fix validation.

The minimal reproducers show that the specific floating-point failures were removed, while the phantom validation shows that the corrected simulator produces accurate quantitative maps.

4.6.1 Why this mattered for qMRI

The timing failure had likely gone unnoticed because it is triggered by long cumulative sequence time, while most examples and tests are much shorter, never reaching the ~ 128 s absolute-time regime where Float64 spacing becomes comparable to the fixed ϵ marker. This threshold is still well within the duration of many clinical MRI acquisitions, which often last several minutes. The largest effect also appears for hard RF pulses with sharp edges: losing one Δt_{rf} sample is a large fraction of a 1 ms rectangular pulse. In practical slice-selective sequences, shaped RF pulses and gradient ramps make the same timing error a smaller perturbation.

A second reason is that most MRI simulation checks are qualitative: a reconstructed image can still look plausible when noise and contrast variation hide small systematic signal errors. qMRI is stricter. The validation compares fitted T_1 values against known phantom ground truth, so a systematic signal error directly appears as parameter bias. This project exposed the failure because RL training needs long sequences and qMRI validation needs quantitative agreement, not just plausible images.

This validation covers the IR-SE sequence family used for the main experiments; other sequence families would require the same timing and no-noise recovery checks before being used for quantitative claims.

4.6.2 Why spoiling was a convincing false lead

Although presented here as a single validation narrative, the investigation happened in two stages. Before either fix, transverse spoiling looked plausible because it reduced the catastrophic 924.5% MAPE failure to 92.6%, suggesting that residual transverse magnetisation might be part of the problem. After the first timing failure was fixed, the remaining error was smaller, around 30–40%, and therefore more consistent with a possible sequence effect. Spoiling had to be rechecked before the second timing bug was isolated. In the intermediate simulator shown in Figures 4.1 to 4.3, however, spoiling increased mean MAPE from 39.4% to 44.0%. After both fixes, spoiled and unspoiled no-noise fits are both sub-percent (0.43% and 0.48% mean MAPE respectively). Spoiling remains important for realistic sequence design, but it was not the cause of this validation failure.

5 Project Plan & Evaluation

Contents

5.1 Project Plan

The project is structured into three distinct phases, transitioning from rapid algorithmic prototyping to high-fidelity validation on clinical hardware. The key challenge will be bridging the gap between Julia (KomaMRI) and Python (PyTorch & RL libraries) and validating the results on the MR-Linac.

Time constraints will also be a big challenge. I will be working on this full-time from the 23rd March to the 12th June, with a 2-week Easter break. This corresponds to 10 full-time weeks, the below plan is based on 12 weeks as I am hoping to make progress on the Environment Setup before the 23rd March alongside my studies. So far, progress has been limited. I have tested KomaMRI and conducted background research into related works.

5.1.1 Phase 1: Prototyping and Environment Setup (Weeks 1–4)

- **Simulator Interfacing:** Develop a Python-based Gymnasium wrapper for the `MRzero` environment to facilitate reinforcement learning.
- **Digital Phantom Design:** Script a digital twin of the **QalibreMD Model 130** phantom, incorporating its 42 spheres with varying T_1, T_2, and PD properties.
- **Milestone:** Successful "sim-to-sim" loop where the RL agent receives a signal from `MRzero` and proposes a sequence change.

5.1.2 Phase 2: RL Training and Simulator Migration (Weeks 5–8)

- **Agent Training:** Train the agent to optimise scan trajectories for the rapid localisation of the Model 130 vials.
- **Simulator Migration:** Port the trained agent to `KomaMRI.jl` using `JuliaCall`. This stage tests the agent's robustness when moving from EPG-based physics to Bloch-based physics.
- **Milestone:** Agent converges on an optimised protocol that accurately identifies vial locations in under 5 minutes.

5.1.3 Phase 3: Validation and qMRI Extraction (Weeks 9–12)

- **MR-Linac Deployment:** Execute the optimised sequences on the MR-Linac using physical phantoms.
- **Project Write-up:** Complete the final BENG report.
- **Milestone:** Comparison of simulated results against experimental phantom data.

5.1.4 Potential Extensions: Advanced qMRI Mapping

If the core objectives are met ahead of schedule, the following extensions will be explored:

- **Autonomous qMRI Mapping:** Beyond mere localisation, the RL reward function will be modified to prioritise the precision of T_1 and T_2 estimates. The agent would "learn" to dwell longer on specific k-space coordinates to maximise the SNR of relaxation time measurements for the QalibreMD spheres.

- **B1+ Inhomogeneity Correction:** Extending the agent's capabilities to identify and compensate for flip-angle variations, a common challenge in 1.5T MR-Linac systems, to improve the accuracy of the resulting qMRI maps.

5.2 Evaluation Plan

The project's success will be evaluated through a rigorous comparison of the RL agent's performance against the ground-truth properties of the **QalibreMD System Standard Model 130** phantom. This phantom is specifically designed for the NIST/ISMRM standard and provides traceable reference values for multi-parameter validation.

5.2.1 Benchmark: The QalibreMD Model 130 Standard

The evaluation will utilize three primary components of the Model 130 phantom to measure the agent's ability to accurately characterize tissue-mimicking materials:

- **T_1 Array:** 14 spheres containing $NiCl_2$ with longitudinal relaxation times ranging from ∼20 ms to 1900 ms.
- **T_2 Array:** 14 spheres containing $MnCl_2$ with transverse relaxation times ranging from ∼5 ms to 550 ms.
- **Proton Density (PD) Array:** 14 spheres with PD values ranging from 5% to 100%.

5.2.2 Quantitative Performance Metrics

The primary objective is to minimize the "Sim-to-Real" gap. Success will be defined by the following metrics:

Metric	**Measurement Description**	**Success Criteria**
Parameter Accuracy	Mean Absolute Percentage Error (MAPE) of T_1, T_2, and PD relative to QalibreMD reference values.	$< 5\%$ error for all arrays.
Spatial Localization	Accuracy in identifying the (x, y) centroids of the 42 spheres within the 5-minute scan window.	Error < 1.0 mm.
Scan Efficiency	Total measurement time required to reach a stable parameter estimate compared to a standard grid-search protocol.	$> 25\%$ reduction in acquisition time.
Signal-to-Noise (SNR)	The agent's ability to maintain sufficient SNR while adapting sequence parameters in real-time.	SNR $\geq$ standard clinical protocol.

Table 5.1: Evaluation benchmarks for the RL agent using the Model 130 phantom. Quantitative success criteria are illustrative estimates.

5.2.3 Experimental Verification Strategy

1. **Phase I (Sim-to-Sim):** The agent's ability to "transfer" its learning from the MRzero EPG simulator to the KomaMRI.jl Bloch simulator will be tested. This ensures the RL strategy is robust to more complex physical artifacts (e.g., off-resonance or B1 inhomogeneities).

2. **Phase II (Sim-to-Real):** The final sequence, optimised by the agent, will be deployed on the MR-Linac. The resulting quantitative maps will be processed using standard fitting algorithms (e.g., `PhantomViewer` or custom Julia scripts) to compare the RL-driven results against the NIST-traceable ground truth.

5.2.4 Contribution to the State-of-the-Art

Current qMRI protocols on the MR-Linac often suffer from long acquisition times or motion artifacts. By demonstrating that an RL agent can autonomously navigate the QalibreMD phantom's parameter space (locating vials and measuring T_1/T_2 simultaneously), this project aims to prove the feasibility of **fully adaptive, autonomous quantitative imaging**.

5.3 Legal and Ethical Matters

The development of a reinforcement learning (RL) framework for MRI sequence optimisation poses several legal and ethical considerations.

5.3.1 Legal Considerations

This project will build upon MRI simulators and RL libraries, so careful consideration of intellectual property (IP) and software licensing is required. In addition to adhering to the terms of use of the MR-Linac and the phantoms used.

- **Intellectual Property and Open Source:** This project will build exclusively on Open Source software. Using both `KomaMRI.jl` and `MRzero` for MRI simulators, `PulSeq` for sequence specifications, and Python's extensive ML libraries (e.g. `PyTorch`). All development will adhere to the original `MIT` or `Apache` licenses. The resulting code and trained RL agents will be documented and shared under similar open-source terms to promote reproducibility in the scientific community.

- **Regulatory Compliance:** While the project is currently in the research phase, the software will interact with an MR-Linac. Since this is a clinical therapeutic device, development must comply with the manufacturer's safety protocols. For example, the RL agent will be unable to request gradient switching rates or radio frequency (RF) power levels that exceed hardware safety limits.
- **Data Protection:** This project will only use physical phantoms (e.g. NIST-traceable phantoms) or their digital twins. As such, no Personal Data or Protected Health Information (PHI) will be processed. Consequently, the project does not fall under the jurisdiction of the General Data Protection Regulation (GDPR).

5.3.2 Ethical Considerations

While testing will be limited to non-biological subjects, the use of a clinical MR-Linac raises ethical considerations.

- **Safety and Hardware Integrity:** A primary ethical concern is the "black box" nature of reinforcement learning. An agent trained to optimise for speed might explore pulse sequences that are physically taxing on the MRI hardware. To address this, the simulation environment includes hard constraints on the Specific Absorption Rate (SAR) and dB/dt limits to ensure that the learned strategies are inherently safe for future clinical translation.
- **Data Integrity and Hallucination:** In the context of AI-driven MRI, there is an ethical risk of the agent generating "hallucinated" features or artifacts to maximise its reward function. This project addresses this by using quantitative MRI (qMRI) phantoms with known ground-truth relaxation times (T_1, T_2), allowing for a rigorous, objective validation of the agent's accuracy.
- **Professional Responsibility:** The use of the MR-Linac for phantom experiments requires adherence to institutional safety guidelines. All experiments will be conducted under the supervision of qualified medical physicists to ensure that the research does not interfere with clinical workflows or compromise the calibration of equipment used for patient treatment.

A

Technical Derivation Notes

This appendix records technical derivations that are useful for understanding the library implementation, but too detailed for the main digital-twin chapter. The first sections derive the fitting models from the Bloch-equation free-precession solutions under two assumptions: RF pulses are treated as instantaneous rotations, and residual transverse magnetisation is spoiled before the next block. Under those assumptions the fitters only need to carry the scalar longitudinal magnetisation M_z between shots.

A.1 Spin-Echo T2 Fit

For a spin echo, the 90° pulse tips equilibrium magnetisation into the transverse plane. The 180° pulse refocuses reversible static dephasing, so the echo magnitude is governed by irreversible T2 decay:

$$|S(\mathrm{TE})| = S_0 e^{-\mathrm{TE}/T_2}. \tag{A.1}$$

Taking logs gives a straight-line model,

$$\log |S| = \log S_0 - \frac{\mathrm{TE}}{T_2}, \tag{A.2}$$

which is the complete model used by `fit_t2_se`.

A.2 Inversion-Recovery T1 Fit

For long-TR inversion recovery, a preparation pulse of angle θ_{inv} leaves $M_z = M_0 \cos\theta_{\text{inv}}$. Recovery for inversion time TI therefore gives

$$M_z(\text{TI}) = M_0 \left[1 - (1 - \cos\theta_{\text{inv}})\, e^{-\text{TI}/T_1}\right]. \tag{A.3}$$

For a textbook 180° inversion, this reduces to $M_0(1 - 2e^{-\text{TI}/T_1})$. Magnitude reconstruction removes the sign around the null point, so the practical conventional estimator fits

$$|S| = \left|A - Be^{-\text{TI}/T_1}\right|. \tag{A.4}$$

This is the model used by `fit_t1_ir`. It is intentionally simple and is only appropriate for the conventional long-TR baseline.

A.3 Finite-TR and Transient IR Models

The RL acquisitions cannot assume long TR or fixed flip angles. For one IR-prepared block, define

$$E_1 = e^{-\text{TI}/T_1}, \qquad E_2 = e^{-(\text{TR}-\text{TI})/T_1}. \tag{A.5}$$

With inversion angle θ_{inv} and excitation angle α_{exc}, the longitudinal recurrence is

$$M_z^{\text{TI}} = (1 - E_1) + \cos(\theta_{\text{inv}}) E_1 M_z^{\text{pre}}, \tag{A.6}$$

$$M_z^{\text{pre}\prime} = (1 - E_2) + \cos(\alpha_{\text{exc}}) E_2 M_z^{\text{TI}}. \tag{A.7}$$

Solving the fixed point gives `steady_state_mz_at_excite`. Iterating the recurrence for the first `Npe` shots and averaging M_z^{TI} gives `transient_mz_at_excite_npe`. This second model matches the actual 2D IR-SE acquisition used in E2: one phase-encode row is acquired per shot, starting from thermal equilibrium.

The measured transverse signal also contains the excitation projection $\sin(\alpha_{\text{exc}})$. When excitation angles vary across blocks, the caller divides magnitudes by this known factor before fitting; otherwise the fitter would confuse flip-angle scaling with T1 contrast.

A.4 Joint T1/T2 Identifiability

Joint T1/T2 fitting becomes possible only when TE varies. The IR-TSE model is

$$|S| = \left| A\, M_z^{\mathrm{exc}}(T_1; \mathrm{TI}, \mathrm{TR}, \theta_{\mathrm{inv}}, \alpha_{\mathrm{exc}}) e^{-\mathrm{TE}/T_2} \right|. \tag{A.8}$$

If TE is constant, the exponential T2 factor is absorbed into the fitted amplitude A, so T2 is unidentifiable. This is why `fit_t1_t2_generalized_ir` is reserved for multi-echo schedules and reports a wide T2 uncertainty for constant-TE data.

A.5 Finite K-space Sampling and Truncation Artefacts

A.5.1 Why Reconstruction Blurs and Rings

Chapter 2 established that an MR image is recovered by applying an inverse Fourier transform to sampled k-space. That Fourier relationship is exact only for infinite k-space. In practice, we acquire a finite $N_{pe} \times N_{fe}$ window, so the reconstructed image is not the true spin density $\rho(\mathbf{r})$ but a degraded version of it. The cleanest way to quantify the degradation is to model finite sampling as the full, infinite k-space multiplied by a rectangular box function: equal to 1 inside the sampled window and 0 outside.

The consequence follows from the convolution theorem: multiplication in one domain is convolution in the other. For a Fourier pair, image space $\leftrightarrow$ k-space,

$$\mathcal{F}\{f \cdot g\} = \mathcal{F}\{f\} * \mathcal{F}\{g\}. \tag{A.9}$$

So multiplying the true k-space by a box is equivalent, in the image domain, to convolving the true image with the Fourier transform of that box. That transform is the system's point-spread function (PSF): each ideal point in the object is smeared into a small PSF-shaped blob, and where the object has a sharp edge those blobs fail to add up cleanly, producing oscillations: Gibbs ringing.

An intuitive way to read the convolution theorem is that a convolution is "slide, multiply, integrate". Under a Fourier transform each shift becomes a phase factor e^{-ikx}, the integral collapses the slide, and only a pointwise product survives. This is also why FFT-based convolution is fast: transform, multiply, transform back.

For a rectangular window of N samples the PSF is the Dirichlet kernel, the periodic sinc:

$$\mathrm{PSF}(x) = \frac{\sin(\pi N x/\mathrm{FOV})}{N \sin(\pi x/\mathrm{FOV})}. \tag{A.10}$$

Reading it as numerator over denominator explains its shape. Let the pixel spacing be $\Delta = \mathrm{FOV}/N$.

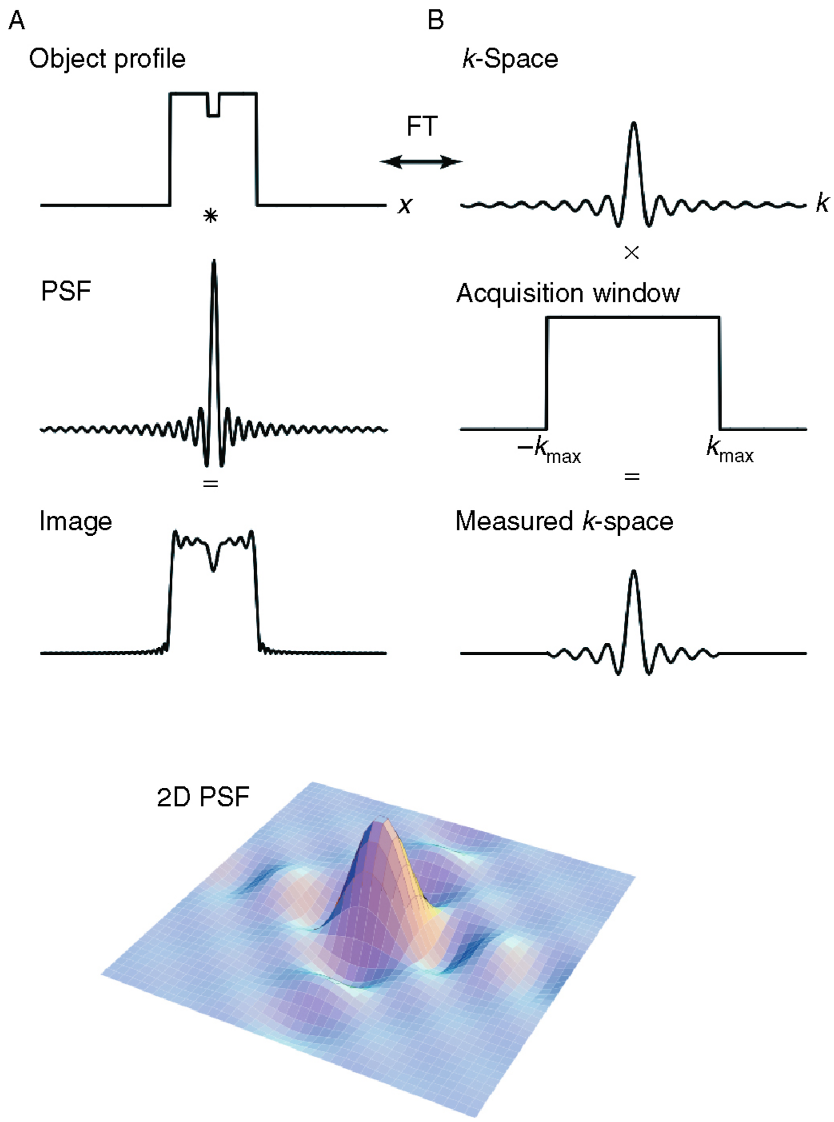


Figure A.1: The point-spread function. Producing a blurred image can be viewed equivalently in the image domain or in k-space. The limited extent of sampling in k-space is described by multiplying the full k-space distribution by a windowing function that removes high spatial frequencies. The resulting image is the convolution of the true image with the Fourier transform of that windowing function, the PSF. Figure adapted from [8].

Quantity	Scaling with N
Main-lobe width, physical units	FOV$/N$, shrinks as $1/N$
Main-lobe width, pixels	1 pixel, always
First side-lobe height	$\approx -21\%$, always
Side-lobe decay	$1/x$, always
Number of side-lobes across the FOV	grows as N

Table A.1: Increasing the sampled matrix size improves physical resolution, but the rectangular-window PSF has the same side-lobe structure in pixel units.

- **Unit peak at $x = 0$.** Both numerator and denominator vanish. L'Hopital's rule gives the ratio $\to 1$, so the centre pixel passes through at full weight. The kernel is normalised; no rescaling of ρ is required.
- **Zeros one pixel apart.** The numerator vanishes when $\pi N x/\mathrm{FOV} = m\pi$, i.e. at $x = m\,\mathrm{FOV}/N = m\Delta$. The main lobe is always exactly one pixel wide.
- **Fixed side-lobes.** The first negative side-lobe is approximately -21% of the peak, or about -13 dB, with a $1/x$ decay. This is set purely by the rectangular window shape.

A useful consequence is that, because the zeros sit on integer pixel offsets, an on-grid point source leaks nothing into its neighbours at the sample points. Ringing only becomes visible at a discontinuity, where the contributions on either side of the edge no longer cancel and the side-lobes ring across neighbouring pixels. This is the classic $\approx 9\%$ Gibbs overshoot on each side of a sharp edge.

A.5.2 Increasing the Matrix Does Not Fix It

The natural reflex is to sample more of k-space, i.e. to use a bigger box. However, a larger N only rescales the ringing; it does not remove it.

In pixel units the PSF shape barely changes: a larger N packs the same ringing into a finer physical scale, giving better resolution, and as $N \to \infty$ the main lobe collapses towards a delta function, recovering the ideal image. But the -21% side-lobe is intrinsic to the rectangular window. To actually suppress ringing, one must change the window shape, not its size. This is apodisation.

A.5.3 Apodisation: The Hamming Window

If a hard rectangular cut is what produces the strong side-lobes, then softening the edge of the k-space window should reduce them. This is apodisation: weighting k-space by a function that tapers smoothly to zero at the edges rather than truncating abruptly. Many window functions exist; a standard choice is the Hamming window,

$$w(n) = 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right), \qquad n = 0, \ldots, N-1. \tag{A.11}$$

which retains full weight near the centre, DC, and tapers to 0.08 at the k-space edges. Because the high spatial frequencies live at those edges, down-weighting them trades resolution for smoothness: the Fourier transform of the Hamming window has a main lobe roughly twice as wide, giving a blurrier image, but side-lobes suppressed from -13 dB to -43 dB, giving far less ringing.

The effect on an actual edge is the case that matters for the phantom, whose water–sphere boundaries are the sharpest features in the scene. The box PSF's side-lobes fail to cancel across the step and produce the $\approx 9\%$ Gibbs overshoot; the Hamming PSF's far weaker side-lobes suppress it, at the price of a softer, wider edge transition.

Two practical remarks close the loop. First, apodisation is a reconstruction-side operation: it is an element-wise multiply applied to the acquired k-space before the inverse FFT, costing a fraction of a

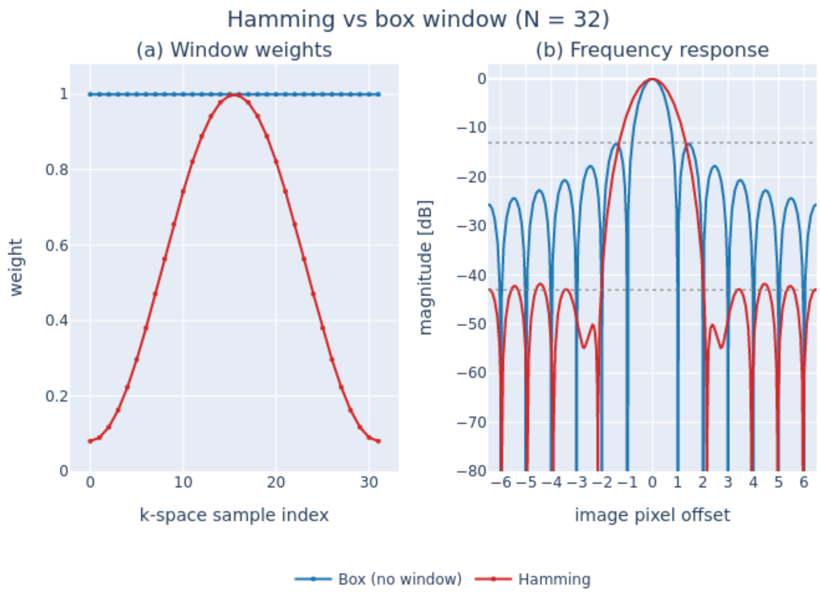


Figure A.2: **Hamming apodisation: window weights and their point-spread response. (a)** K-space weighting applied before reconstruction for $N = 32$ phase-encode lines. The box window, i.e. no apodisation, weights every line equally at 1.0; the Hamming window $0.54 - 0.46\cos(2\pi n/(N-1))$ tapers smoothly to 0.08 at the k-space edges, retaining full weight only near the centre. **(b)** Corresponding point-spread functions, shown as magnitude in decibels normalised to the main-lobe peak. The box window's abrupt truncation produces -13 dB side-lobes that ring across neighbouring pixels; the Hamming taper suppresses these to -43 dB at the cost of an approximately $2\times$ wider main lobe, i.e. reduced spatial resolution.

millisecond and entirely independent of the expensive Bloch simulation. It can therefore be toggled freely without re-acquiring or re-simulating. Second, it must not be confused with zero-padding. Padding k-space with zeros before the FFT interpolates the image onto a finer grid but adds no new measured frequencies, so it does not remove ringing. It only resamples the same Dirichlet PSF more densely. Genuine suppression requires changing the window shape, which is exactly what the phantom library's Hamming option does, and the mechanism here is what underpins its use against the coarse-water truncation artefact in Chapter 3.

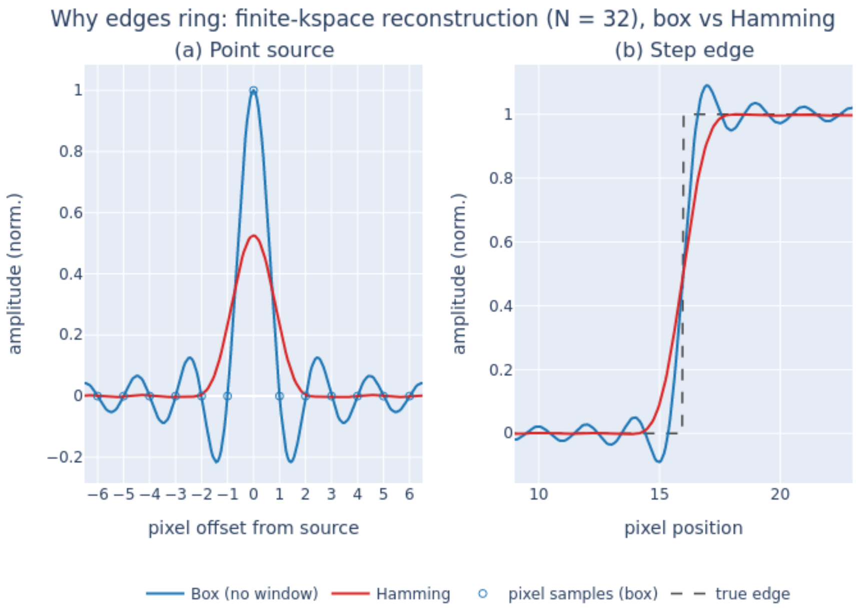


Figure A.3: **Why finite k-space sampling rings, and how Hamming apodisation suppresses it.** One-dimensional reconstruction from a finite acquisition of $N = 32$ phase-encode lines, comparing no window, or box truncation, with a Hamming window. **(a)** Point-source response: the reconstruction PSF. The box truncation yields a Dirichlet, sinc-like PSF whose zero-crossings fall exactly on integer pixel offsets. An on-grid point leaks nothing into neighbouring pixels at the sample points. The Hamming PSF has an approximately 2× wider main lobe, nonzero at ±1 px, so it blurs adjacent pixels, and a lower peak reflecting the window's DC attenuation. **(b)** The same two PSFs convolved with a step edge. The box PSF's −13 dB side-lobes fail to cancel across the discontinuity, producing Gibbs ringing with ≈ 9% overshoot on both sides. The Hamming window's −43 dB side-lobes suppress this ringing, at the cost of a softer, wider edge transition. Equivalently, panel (a) is the PSF and panel (b) is that PSF convolved with an edge: the point-spread function explains the edge artefact.

Bibliography

[1] W. C. R. Fund, *UK cancer statistics*, en-GB. [Online]. Available: https://www.wcrf.org/preventing-cancer/cancer-statistics/uk-cancer-statistics/ (visited on 01/21/2026).

[2] R. Atun, D. A. Jaffray, and M. B. e. a. Barton, "Expanding global access to radiotherapy," *The Lancet Oncology*, vol. 16, no. 10, pp. 1153–1186, Sep. 2015. DOI: 10.1016/s1470-2045(15)00222-3. [Online]. Available: https://doi.org/10.1016/s1470-2045(15)00222-3.

[3] X. Liu, Z. Li, and Y. Yin, "Clinical application of MR-Linac in tumor radiotherapy: A systematic review," *Radiation Oncology*, vol. 18, no. 1, p. 52, Mar. 2023, ISSN: 1748-717X. DOI: 10.1186/s13014-023-02221-8. [Online]. Available: https://doi.org/10.1186/s13014-023-02221-8.

[4] J. de Leon, T. Twentyman, M. Carr, M. Jameson, and V. Batumalai, "Optimising the mr-linac as a standard treatment modality," *Journal of Medical Radiation Sciences*, vol. 70, no. 4, pp. 491–497, 2023. DOI: https://doi.org/10.1002/jmrs.712. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/jmrs.712. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/jmrs.712.

[5] R. Heckel, M. Jacob, A. Chaudhari, O. Perlman, and E. Shimron, "Deep learning for accelerated and robust MRI reconstruction," *Magnetic Resonance Materials in Physics, Biology and Medicine*, vol. 37, no. 3, pp. 335–368, Jul. 2024, ISSN: 1352-8661. DOI: 10.1007/s10334-024-01173-8. [Online]. Available: https://doi.org/10.1007/s10334-024-01173-8.

[6] D. B. Plewes and W. Kucharczyk, "Physics of mri: A primer," *Journal of Magnetic Resonance Imaging*, vol. 35, no. 5, pp. 1038–1054, 2012. DOI: https://doi.org/10.1002/jmri.23642. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/jmri.23642. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/jmri.23642.

[7] R.-J. M. van Geuns, P. A. Wielopolski, H. G. de Bruin, *et al.*, "Basic principles of magnetic resonance imaging," *Progress in Cardiovascular Diseases*, vol. 42, no. 2, pp. 149–156, 1999. DOI: https://doi.org/10.1016/S0033-0620(99)70014-9. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0033062099700149.

[8] R. B. Buxton, "Mapping the mr signal," in *Introduction to Functional Magnetic Resonance Imaging: Principles and Techniques*. Cambridge University Press, 2009, pp. 205–231.

[9] K. J. Layton, S. Kroboth, F. Jia, *et al.*, "Pulseq: A rapid and hardware-independent pulse sequence prototyping framework," *Magnetic Resonance in Medicine*, vol. 77, no. 4, pp. 1544–1552, 2017. DOI: https://doi.org/10.1002/mrm.26235. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/mrm.26235. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/mrm.26235.

[10] C. Castillo-Passi, R. Coronado, G. Varela-Mattatall, C. Alberola-López, R. Botnar, and P. Irarrazaval, "Komamri.jl: An open-source framework for general mri simulations with gpu acceleration," *Magnetic Resonance in Medicine*, vol. 90, no. 1, pp. 329–342, 2023. DOI: https://doi.org/10.1002/mrm.29635. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/mrm.29635. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/mrm.29635.

[11] A. Loktyushin, K. Herz, N. Dang, *et al.*, "Mrzero - automated discovery of mri sequences using supervised learning," *Magnetic Resonance in Medicine*, vol. 86, no. 2, pp. 709–724, 2021. DOI: https://doi.org/10.1002/mrm.28727. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/mrm.28727. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/mrm.28727.

[12] B. Mehta, S. Coppo, D. F. McGivney, *et al.*, "Magnetic resonance fingerprinting: A technical review," *Magnetic Resonance in Medicine*, vol. 81, no. 1, pp. 25–46, 2019. DOI: 10.1002/mrm.27403.

[13] S. P. Jordan, S. Hu, I. Rozada, *et al.*, "Automated design of pulse sequences for magnetic resonance fingerprinting using physics-inspired optimization," *Proceedings of the National Academy of Sciences*, vol. 118, no. 40, e2020516118, 2021. DOI: 10.1073/pnas.2020516118. eprint: https://www.pnas.org/content/118/40/e2020516118.full.pdf. [Online]. Available: https://www.pnas.org/content/118/40/e2020516118.

[14] A. Ocanto, L. Torres, M. Montijano, *et al.*, "MR-LINAC, a New Partner in Radiation Oncology: Current Landscape," *Cancers*, vol. 16, no. 2, p. 270, Jan. 2024, ISSN: 2072-6694. DOI: 10.3390/cancers16020270. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10813892/ (visited on 01/20/2026).

[15] A. Kuisma, I. Ranta, J. Keyriläinen, *et al.*, "Validation of automated magnetic resonance image segmentation for radiation therapy planning in prostate cancer," *Physics and Imaging in Radiation Oncology*, vol. 13, pp. 14–20, 2020, ISSN: 2405-6316. DOI: https://doi.org/10.1016/j.phro.2020.02.004. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S240563162030004X.

[16] A. D. Elster, *Magnetism*, en. [Online]. Available: http://mriquestions.com/ai-in-clinical-mri.html (visited on 01/20/2026).

[17] B. Zhu, J. Liu, N. Koonjoo, B. R. Rosen, and M. S. Rosen, "Automated pulse sequence generation (autoseq) using bayesian reinforcement learning in an mri physics simulation environment," in *Proceedings of the Joint Annual Meeting ISMRM-ESMRMB*, 2018, p. 0438.

[18] I. Beracha, A. Seginer, and A. Tal, "Adaptive model-based magnetic resonance," *Magnetic Resonance in Medicine*, vol. 90, no. 3, pp. 839–851, 2023. DOI: `https://doi.org/10.1002/mrm.29688`. eprint: `https://onlinelibrary.wiley.com/doi/pdf/10.1002/mrm.29688`. [Online]. Available: `https://onlinelibrary.wiley.com/doi/abs/10.1002/mrm.29688`.

[19] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. [Online]. Available: `http://incompleteideas.net/book/the-book-2nd.html`.

[20] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Advances in Neural Information Processing Systems*, vol. 12, 1999. [Online]. Available: `https://papers.nips.cc/paper/1713-policy-gradient-methods-for-reinforcement-learning-with-function-approximation`.

[21] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, pp. 229–256, 1992. DOI: `10.1007/BF00992696`. [Online]. Available: `https://doi.org/10.1007/BF00992696`.

[22] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *International Conference on Learning Representations*, 2016. DOI: `10.48550/arXiv.1506.02438`. arXiv: `1506.02438`. [Online]. Available: `https://arxiv.org/abs/1506.02438`.

[23] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proceedings of the 32nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 37, PMLR, 2015, pp. 1889–1897. [Online]. Available: `https://proceedings.mlr.press/v37/schulman15.html`.

[24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017. DOI: `10.48550/arXiv.1707.06347`. arXiv: `1707.06347`. [Online]. Available: `https://arxiv.org/abs/1707.06347`.

[25] S. Walker-Samuel, "Using deep reinforcement learning to actively, adaptively and autonomously control a simulated mri scanner," in *Proceedings of the Joint Annual Meeting ISMRM-ESMRMB*, 2019, p. 0473. [Online]. Available: `https://cds.ismrm.org/protected/19MProceedings/PDFfiles/0473.html`.